Seraph Protocol — Technical Manual

Comprehensive engineering manual for the Seraph Protocol: Modular privacy continuum combining a Monero-inspired base layer with dual private EVMs.

- Part I: Foundations & Cryptography
  - Chapter 0: Protocol Overview & The Privacy Continuum
  - Chapter 1: Monero Heritage & Evolution
  - Chapter 2: Seraphis, Jamtis & Research Horizons
  - Chapter 3: Formal Cryptography & Primitives
  - Chapter 4: ZK Circuit Architecture & Prover Pipeline
- Part II: Core Architecture & Networks
  - Chapter 5: Base Layer Protocol & State Machine
  - Chapter 6: Dual Private EVMs
  - Chapter 7: Bridges & Cross-Chain Architecture
  - Chapter 8: Relayers & Gas Abstraction
  - Chapter 9: Sequencer Architecture & MEV Shielding
- Part III: Applications & Tokenomics
  - Chapter 10: Wallets, Scanning & Key Recovery
  - Chapter 11: First-Party Privacy Modules
  - Chapter 12: Ecosystem, DEX & CEX Integrations
  - Chapter 13: SER Tokenomics, Supply & Burns
  - Chapter 14: Presale Architecture & Vesting
  - Chapter 15: Launchpad Mechanics & Creator Playbook
- Part IV: Operations, Security & Reference
  - Chapter 16: Decentralized Governance & Private Ballots
  - Chapter 17: Security Model, Threat Vectors & Pentest Matrix
  - Chapter 18: Emergency Procedures & Circuit Breakers
  - Chapter 19: Protocol Roadmap & Milestones
  - Chapter 20: Glossary of Terms
  - Chapter 21: Contributing, Standards & Dev Setup

Part I: Foundations & Cryptography

Privacy Continuum L0-L3, Monero heritage, Seraphis research horizons, formal cryptography, and ZK proving benchmarks.

Chapter 0: Protocol Overview & The Privacy Continuum

Chapter 0: Protocol Overview & The Privacy Continuum

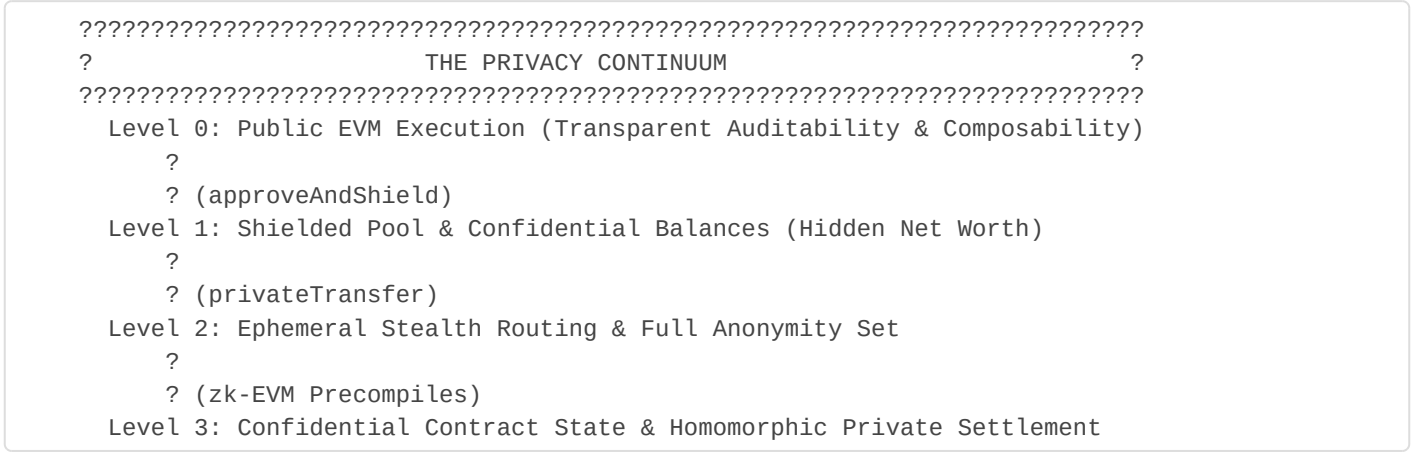
0.1 Introduction: What Seraph Is

Seraph is a modular privacy protocol engineered to resolve the fundamental trilemma between **cryptographic privacy**, **programmable smart contracts**, and **regulatory auditability**.

Historically, decentralized ledger architectures have been split into two mutually exclusive camps:

1. **UTXO-based Privacy Networks** (such as Monero): Offering peerless anonymity sets, confidential transaction values, and stealth unlinkability, but lacking Turing-complete contract execution and multi-token composability.
2. **Account-based Smart Contract Platforms** (such as Ethereum and EVM rollups): Offering rich programmable composability, decentralized finance (DeFi), and autonomous application development, but exposing complete transaction graphs, account balances, and interaction histories to global surveillance and predatory MEV bots.

Seraph bridges this divide through a **four-level Privacy Continuum** anchored by a high-throughput, Monero-inspired base layer running alongside dual confidential Ethereum Virtual Machines (Seraph EVM and XMR EVM).



0.2 The Privacy Continuum: Levels 0 to 3

Seraph recognizes that privacy is not a binary toggle, but a continuum where different economic activities demand different privacy assurances:

Level	Name	Visibility	State Architecture	Primary Use Cases
Level 0	**Public EVM**	Sender, recipient, and amount fully public	Standard ERC-20 (`SERAPH.sol`) & EVM balance storage	DEX market liquidity, launchpad token discovery, public treasury accounting
Level 1	**Shielded Pool**	Balances hidden; deposit/withdraw events public	Merkle tree of Pedersen commitments (`PrivacyPool.sol`)	Long-term balance protection, wealth storage without exposing wallet balances
Level 2	**Full Stealth**	Sender, recipient, amount, and asset completely obfuscated	Depth-20 Incremental Merkle tree (2^{20} capacity) + Nullifier registry	Private P2P payments, payroll settlement, dark pool order placement
Level 3	**Confidential Contracts**	Contract logic executed with encrypted state and zero-knowledge inputs	Dual Private EVMs with homomorphic precompiles & ZK proof verifiers	Private lending/CDP (`CollateralVault`), AI usage billing (`UsageVault`), sealed ballots

When Each Level Applies

- **Choose Level 0** when bootstrapping public liquidity on automated market makers (AMMs), interacting with third-party protocol contracts (e.g., Uniswap v3, Aave), or demonstrating provable balance solvency.
- **Choose Level 1** when storing capital long-term to prevent targeted spear-phishing and balance scraping, without disconnecting from the EVM ecosystem.
- **Choose Level 2** when conducting sensitive financial settlements between counterparties where neither party's transaction graph should be linked.
- **Choose Level 3** when executing stateful multiparty logic—such as streaming payments to autonomous AI agents, algorithmic market makers with hidden depth, or institutional credit facilities.

0.3 Who the Docs Are For

This documentation suite is organized into distinct tracks tailored to four primary stakeholder groups:

1. End Users & Privacy Advocates

- Understand the mechanics of shielding and unshielding tokens.

- Manage desktop, mobile, and web wallets safely.
- Learn how view keys protect privacy while allowing selective compliance disclosure.

2. Smart Contract & Frontend Developers

- Build privacy-preserving decentralized applications using standard Solidity.
- Integrate the `@seraph-protocol/sdk` TypeScript client and Rust base-layer bindings.
- Utilize on-chain precompiles for confidential state management, stealth addresses, and zero-knowledge verification.

3. Validators, Relayers & Node Operators

- Run full nodes and lightweight verification clients.
- Operate bonded relayers on the Cross-EVM Bridge and earn protocol routing fees.
- Participate in sequencer clusters and consensus validation.

4. Token Holders & Ecosystem Creators

- Review SER tokenomics, supply caps, and automatic fee burn mechanics.
- Participate in DAO governance proposals and confidential voting ballots.
- Deploy fair-launch bonding curve tokens with custom multi-way fee distributions using the Seraph Launchpad.

Chapter 1: Monero Heritage & Evolution

Chapter 1: Monero Heritage & Evolution

1.1 The Genesis of Decentralized Privacy

Monero (XMR) demonstrated to the world that decentralized currencies could achieve true fungibility by concealing transaction senders, recipients, and amounts. Seraph directly inherits the cryptographic ethos of Monero while evolving its computational foundations to support global-scale transaction throughput and Turing-complete smart contract execution.

1.2 Ring Signatures and Why Seraph Moves Past Them

Monero historically pioneered **Linkable Spontaneous Anonymous Group (LSAG)** and **Multilayered Linkable Spontaneous Anonymous Group (MLSAG)** signatures, followed by **CLSAG** (Compact Linkable Spontaneous Anonymous Group) signatures.

In a ring signature scheme:

- The real signer mixes their input with $n-1$ decoy outputs selected randomly from the blockchain history.
- An observer knows that *one* member of the ring spent the output, but cannot mathematically determine which one.
- In current Monero, the ring size is fixed at $n = 16$.

Limitations of Ring Signatures

1. **Bounded Anonymity Set**: A ring size of 16 provides an anonymity set of only 16 possible signers per input.
2. **Statistical and Temporal Clustering (EAE Attacks)**: Sophisticated chain analysis firms use temporal heuristic clustering, spend-time analysis, and "effective ring size" reduction when known outputs are spent.
3. **Linear On-Chain Footprint**: Verification complexity and signature byte size scale linearly with ring size: $$\mathcal{O}(n)$$ Increasing the ring size to 1,024 or 65,536 would bloat the blockchain beyond consumer hardware verification capacity.

The Seraph Shift: Incremental Merkle Trees & Zero-Knowledge Proofs

Seraph transitions from small decoy rings to **full-membership zero-knowledge proofs** over an incremental Merkle tree:

- **Anonymity Set**: Encompasses all unspent notes currently in the Merkle tree ($2^{20} = 1,048,576$ notes per pool).
- **Proof Size**: $\mathcal{O}(1)$ constant-size Groth16/PLONK proofs (128–256 bytes) verified in constant time (≈ 3 ms) on-chain.
- **Immunity to Decoy Poisoning**: There are no decoys; the proof proves membership in the entire accumulated tree without revealing which leaf is being spent.

1.3 Stealth Addresses and One-Time Keys

To preserve recipient privacy, Seraph adopts the Dual-Key Stealth Address Protocol (DKSAP). A recipient publishes a static public identity consisting of two elliptic curve points:

- **Scan Public Key**: $A = aG$
- **Spend Public Key**: $B = bG$

When a sender constructs a payment to (A, B):

1. Sender generates an ephemeral private key $r \xleftarrow{$} \mathbb{Z}_q$ and computes the ephemeral public key: $$R = rG$$
2. Sender derives a shared Diffie-Hellman secret: $$S = rA = r(aG) = a(rG) = aR$$
3. Sender hashes the shared secret to derive a scalar tweak: $$t = \mathcal{H}_s(S)$$
4. Sender computes the one-time destination address: $$P = tG + B = (\mathcal{H}_s(rA) + b)G$$

Recipient Scanning & One-Time Private Key Recovery

The recipient, monitoring incoming ledger commitments, computes: $$S' = aR = a(rG) = r(aG) = S$$ $$t' = \mathcal{H}_s(S')$$ $$P' = t'G + B$$

If $P' == P$, the note belongs to the recipient! The recipient derives the corresponding one-time spend key: $$x = t' + b = \mathcal{H}_s(aR) + b$$ Notice that: $$xG = (\mathcal{H}_s(aR) + b)G = P$$ Only the holder of the spend private key b can construct x and spend the note.

1.4 RingCT and Confidential Amounts

Prior to RingCT (Ring Confidential Transactions, Shen Noether 2015), Monero transactions exposed raw numerical values. RingCT introduced **Pedersen Commitments** to hide values while preserving mathematical balance:

$$C = vH + rG$$

where:

- v is the numeric transfer amount.
- r is a secret blinding factor.
- G and H are independent generators of the elliptic curve such that $\log_G(H)$ is computationally infeasible to determine (nothing-up-my-sleeve point).

Homomorphic addition allows the ledger to verify that the sum of input commitments equals the sum of output commitments plus fees without revealing v: $$\sum C_{\text{in}} - \sum C_{\text{out}} - \text{Fee} \cdot H = 0$$

1.5 Bulletproofs and Bulletproofs++

In confidential value systems, an attacker could mint infinite counterfeit funds by committing to a negative value: $$-1 \equiv q - 1 \pmod q$$ To prevent this, every confidential output must include a **Range Proof** proving: $$v \in [0, 2^{64}-1]$$

Monero transitioned from Borromean ring range proofs (multi-kilobyte) to **Bulletproofs** (Bootle et al., 2016; Bünz et al., 2018), which slashed transaction size by over 80% via logarithmic inner-product arguments: $$\mathcal{O}(\log m)$$ Bulletproofs++ further reduced proof size and verification times.

In Seraph, range checks are integrated directly into the arithmetic circuits of our zero-knowledge provers, enabling native verification inside the EVM with zero bloat.

1.6 Key Images and Double-Spend Prevention

To ensure that an obfuscated note cannot be spent twice without revealing which note was spent, Monero uses **Key Images**, and Seraph uses **Nullifiers**.

In Monero, for a one-time key $P = xG$, the key image is defined as: $$I = x \mathcal{H}_p(P)$$

Because x is unique to that one-time note, I is deterministic and unique. If an attacker attempts to spend the same note twice, the ledger detects the duplicate I and rejects the transaction immediately.

Seraph adapts this principle into a deterministic zero-knowledge nullifier: $$\mathcal{N} = \text{Keccak256}(\text{spendKey} \mathbin{\Vert} \text{leafIndex})$$ This ensures absolute double-spend immunity with $O(1)$ constant-time lookup in the `nullified` mapping.

1.7 Full-Chain Membership Proofs (FCMP++)

Monero research has actively pursued **Full-Chain Membership Proofs (FCMP++)**, replacing decoy rings with polynomial commitment schemes (such as Curve Trees) over the entire chain history.

In Seraph:

- The production base layer and EVM privacy pool achieve full-chain membership today via **incremental Merkle trees of depth 20** with $O(1)$ zero-knowledge SNARK membership verification.
- **FCMP++ and Curve Tree primitives** are tracked as active research horizons for future trustless cross-curve recursion between Curve25519 and BN254.

Chapter 2: Seraphis, Jamtis & Research Horizons

Chapter 2: Seraphis, Jamtis & Research Horizons

2.1 Technical Distinction: Production Architecture vs. Research Horizons

> [!IMPORTANT] **Architectural Clarity**: The current production deployment of Seraph Protocol (including our Base Sepolia contracts, Rust base layer, and TypeScript SDK) operates on an **incremental Merkle tree of depth 20 ($2^{20} = 1,048,576$ notes)** with deterministic nullifier derivation and Groth16/PLONK zero-knowledge spend verification.
>
> **Seraphis**, **Jamtis**, and **FCMP++** represent next-generation cryptographic upgrade paths on our long-term research roadmap. They are not active in the current Phase 0 testnet contracts, but form the theoretical blueprint for Seraph Phase 4.

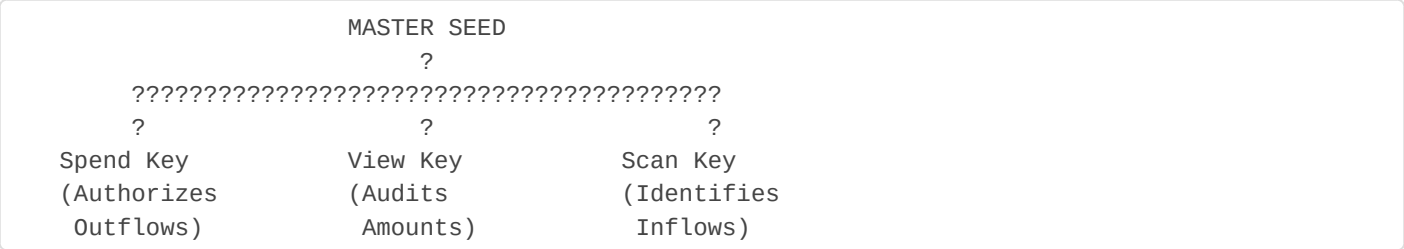
2.2 The Seraphis Model: Enotes and Separation of Concerns

Seraphis is a next-generation transaction protocol designed by koe for Monero. It fundamentally redesigns the UTXO concept into modular cryptographic objects called **electronic notes (enotes)**.

In legacy CryptoNote/Monero protocols, a single private key is entangled with:

1. Detecting incoming funds (scanning).
2. Proving balance ownership.
3. Signing spending authorization.

Seraphis separates these capabilities into distinct, non-overlapping keys:



The Structure of an Enote

An enote in Seraphis consists of:

1. **Value Commitment**: A Pedersen commitment $C = vH + rG$ concealing the value.
2. **One-Time Public Key**: Derived via stealth address protocols.
3. **Key Image Generator**: An elliptic curve point used specifically for nullifier creation, disentangled from the one-time address.

2.3 Jamtis: Hierarchical Address and Wallet Key Structure

Jamtis is the comprehensive wallet addressing standard designed to accompany Seraphis. Jamtis addresses are divided into specific key components:

$$\text{Jamtis Address} = (\mathcal{K}_{\text{spend}}, \mathcal{K}_{\text{view}}, \mathcal{K}_{\text{scan}})$$

Advantages of Jamtis Wallet Hierarchies:

- **Fast Ledger Scanning**: A lightweight client or hardware wallet can delegate the scan key to a cloud scanning node without giving that node the ability to see transaction values or spend funds.
- **Selective Audit Keys**: An auditor can be given a read-only audit view key that sees transaction amounts for tax reconciliation, but cannot initiate transfers.
- **Delegated Proof Generation**: Zero-knowledge proof generation can be offloaded to untrusted high-performance server clusters using an ephemeral proving delegation key.

2.4 Balance and Membership Proofs in Seraphis

In Seraphis, a transaction is structured as an algebraic balance proof across three components:

1. **Membership Proofs**: Each input enote proves that it belongs to the valid set of unspent transaction outputs.

2. **Value Balance Proof**: Proves that: $$\sum_{i} C_{\text{in}}^{(i)} - \sum_{j} C_{\text{out}}^{(j)} - \text{Fee} \cdot H = 0$$
3. **Range Proofs**: Bulletproofs++ proving that each output value is non-negative and strictly bounded within $[0, 2^{64}-1]$.

2.5 How Seraph Adapts These Principles to the Base Layer

Seraph takes the core mathematical insights of Seraphis—particularly **modular key separation** and **unlinkable enote structures**—and adapts them into a high-performance EVM-compatible framework:

1. Dual View Keys in Production

Seraph ships today with separate **Incoming View Keys** (for scanning unspent notes) and **Outgoing View Keys** (for generating verifiable compliance disclosure packages).

```
// Production SDK Key Separation (sdk/typescript/src/index.ts)
const keys = deriveWalletKeys(spendKey);
// keys.spendKey -> authorizes nullifier consumption
// keys.viewKey  -> scans and decrypts incoming note memos
// keys.pubkey   -> static identity for stealth address derivation
```

2. State-Tree Note Commitments

Rather than requiring global linear scans across the entire chain history, Seraph organizes all commitments into an indexed Merkle tree where inclusion proofs can be verified on-chain in constant time ($O(1)$) with zero storage bloat.

3. Native Multi-Asset Isolation

Unlike legacy systems designed around a single native asset, Seraph notes natively incorporate an `AssetId` domain tag: $$\text{Commitment} = \text{Keccak256}(\text{AssetId} \mathbin{\Vert} \text{Amount} \mathbin{\Vert} \text{RecipientPubkey} \mathbin{\Vert} \text{Blinding})$$

This allows SER tokens, native XMR, and custom confidential assets to share the exact same anonymity pool without cross-asset substitution risk.

Chapter 3: Formal Cryptography & Primitives

Chapter 3: Formal Cryptography & Primitives

3.1 Mathematical Notation & Elliptic Curve Parameters

Seraph uses a combination of two primary elliptic curve groups:

1. **Ed25519 / Curve25519**: Used on the high-throughput base layer for stealth addresses, signature verification, and view-key encryption.
   - Prime field: $p = 2^{255} - 19$
   - Base point generator: G
   - Curve equation: $-x^2 + y^2 = 1 - \frac{121665}{121666} x^2 y^2$
2. **BN254 (alt_bn128)**: Used for on-chain zero-knowledge SNARK proof verification inside the Ethereum Virtual Machine via EVM precompiles at `0x06`, `0x07`, and `0x08`.
   - Prime field: $q = 21888242871839275222246405745257275088548364400416034343698204186575808495617$
   - Curve order: $r = 21888242871839275222246405745257275088696311157297823662689037894645226208583$

Common Symbols

- $\mathbb{F}_q$: The scalar field of order q.
- $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$: The pairing groups for BN254.
- $x \xleftarrow{$} \mathbb{F}_q$: Uniform random sampling from the field.
- $\mathcal{H}_s(\cdot)$: Cryptographic hash function mapping arbitrary byte strings to a field scalar in $\mathbb{F}_q$.
- $\mathcal{H}_p(\cdot)$: Hash function mapping arbitrary data to a point on the elliptic curve group $\mathbb{G}$.

3.2 Pedersen Commitments

A Pedersen commitment binds the committer to a specific value v without revealing that value to observers.

Construction

Let G and H be two independent generator points of the elliptic curve such that $\log_G(H)$ is unknown: $$C = vH + rG$$ where:

- $v \in [0, 2^{64}-1]$ is the confidential amount.
- $r \in \mathbb{F}_q$ is the uniform random blinding factor.

Security Properties

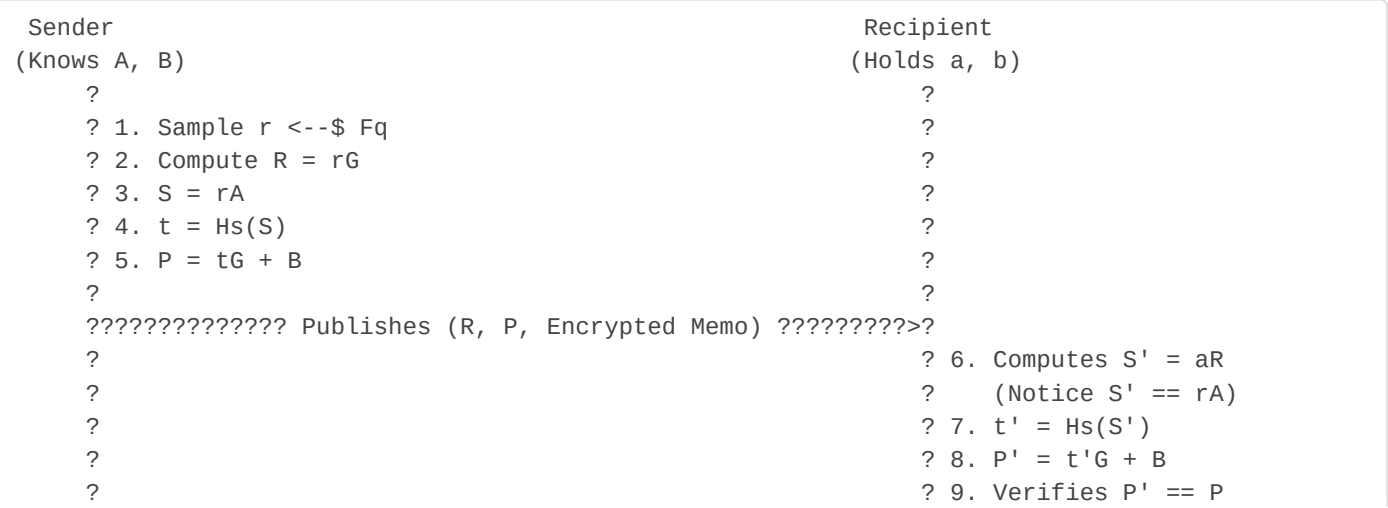
1. **Information-Theoretic Hiding**: For any value v, and for any chosen commitment C, there exists a valid blinding factor r such that $C = vH + rG$. Therefore, an observer with infinite computational power learns zero information about v.
2. **Computational Binding**: Assuming the discrete logarithm problem is hard, it is computationally infeasible to find (v, r) and (v', r') such that $v \neq v'$ and: $$vH + rG = v'H + r'G$$ If an adversary could find such values, they could calculate the discrete logarithm: $$\log_G(H) = \frac{r' - r}{v - v'}$$

Homomorphic Linearity

Pedersen commitments support homomorphic addition: $$C_1 + C_2 = (v_1 H + r_1 G) + (v_2 H + r_2 G) = (v_1 + v_2)H + (r_1 + r_2)G$$

3.3 Stealth Address Derivation

Seraph uses the Dual-Key Stealth Address Protocol (DKSAP) to ensure that transactions sent to a recipient can never be linked to that recipient's public wallet address.



```
    ?                                                    ?10. Recovers x = t' + b
```

Because $xG = (t' + b)G = t'G + bG = P$, only the recipient possessing the private spend key b can authorize transfers from P.

3.4 Key Images and Nullifiers

To ensure privacy while preventing double-spending, Seraph uses two deterministic nullifier constructions:

1. Base Layer Key Image

For a one-time key $P = xG$: $$I = x \mathcal{H}_p(P)$$ If an adversary attempts to spend the note twice, they are forced to use the exact same private key x, producing an identical key image I.

2. EVM Privacy Pool Nullifier

In the `PrivacyPool.sol` smart contract, each private note has a unique nullifier: $$\mathcal{N} = \text{Keccak256}(\text{spendKey} \mathbin{\Vert} \text{leafIndex})$$ When a note is spent via `privateTransfer` or `unshield`:

1. The contract checks `require(!nullified[nullifier], "nullifier already spent")`.
2. The contract sets `nullified[nullifier] = true`.
3. The zero-knowledge verifier confirms that the nullifier corresponds to the note at `leafIndex` proven in the Merkle tree.

3.5 The Balance Equation with Protocol Fees

For any transaction consuming m input notes and generating n output notes with a public protocol fee f, the balance equation is:

$$\sum_{i=1}^m v_{\text{in}}^{(i)} = \sum_{j=1}^n v_{\text{out}}^{(j)} + f$$

Expressed in Pedersen commitments: $$\sum_{i=1}^m C_{\text{in}}^{(i)} - \sum_{j=1}^n C_{\text{out}}^{(j)} - f \cdot H = \left(\sum_{i=1}^m r_{\text{in}}^{(i)} - \sum_{j=1}^n r_{\text{out}}^{(j)}\right) G$$

Let $\Delta r = \sum r_{\text{in}} - \sum r_{\text{out}}$. The transaction prover provides a signature proving knowledge of the private scalar Δr with respect to the public key point $\Delta r G$.

3.6 Range Proofs

Without range proofs, an adversary could spend 10 SER and create two output notes of 15 SER and -5 SER: $$15 + (-5) = 10$$ Because arithmetic in $\mathbb{F}_q$ is modular: $$-5 \equiv q - 5 \pmod q$$ A negative number is represented as a massive positive scalar, minting counterfeit tokens.

Seraph solves this by enforcing: $$v \in [0, 2^{64}-1]$$ This is proven either via logarithmic **Bulletproofs++** inner-product arguments on the base layer, or via zero-knowledge arithmetic range constraints within the SNARK circuits on the EVM.

3.7 Viewing Keys & Selective Compliance Disclosure

Seraph addresses the compliance dilemma through asymmetric key decoupling:

1. Incoming View Key ($\mathcal{K}_{\text{in}}$)

- Derived as $\mathcal{K}_{\text{in}} = \text{Keccak256}(\text{spendKey} \mathbin{\Vert} \text{"seraph_viewkey_derive"})$.
- Used by client wallets to scan the global Merkle tree and decrypt incoming note amounts and memos.
- Cannot spend funds or create transactions.

2. Outgoing View Key ($\mathcal{K}_{\text{out}}$)

- Authorizes generation of cryptographically signed compliance packages.
- Allows users to provide selective disclosure to accountants, auditors, or tax authorities proving:
  - Exact unshielded amounts.
  - Destination public addresses.
  - Timestamps and transaction hashes.
  - Authenticity verified without revealing other wallet balances.

Chapter 4: ZK Circuit Architecture & Prover Pipeline

Chapter 4: ZK Circuit Architecture & Prover Pipeline

4.1 Overview of the Proof System

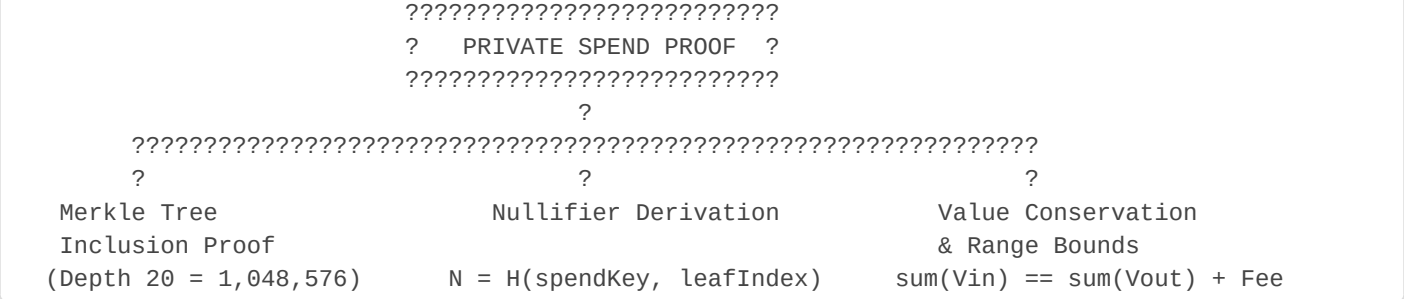
The zero-knowledge proving subsystem of Seraph converts state transitions into rank-1 constraint systems (R1CS) and algebraic intermediate representations (AIR) verifiable within the Ethereum Virtual Machine (EVM).

Seraph utilizes two complementary proving backends:

1. **Groth16 (BN254)**: Deployed on the EVM for maximum gas efficiency. Groth16 yields 128-byte proofs and requires only 3 pairing operations during on-chain verification, consuming approximately 190,000 gas.
2. **UltraPLONK / Halo2**: Used for complex confidential contract state transitions on the private EVMs, offering custom gates, lookup tables, and universal setup parameters.

4.2 The Private Spend Circuit

The core circuit governing `PrivacyPool.sol` enforces four simultaneous constraints for every private spend:



Circuit Constraints Specification:

1. **Merkle Inclusion Proof**:
   - Given a historic root $R \in \text{roots}[0..99]$ and a note commitment C, prove that C exists at `leafIndex` along the path $\mathbf{p} = [p_0, p_1, \dots, p_{19}]$: $$h_0 = C$$ $$h_{i+1} = \text{HashPair}(h_i, p_i) \quad \text{for } i \in [0, 19]$$ $$\text{Enforce: } h_{20} == R$$
2. **Nullifier Derivation**:
   - The prover must prove knowledge of the private `spendKey` such that: $$\mathcal{N} = \text{Keccak256}(\text{spendKey} \mathbin{\Vert} \text{leafIndex})$$
   - Because `spendKey` is never revealed, the nullifier cannot be forged.
3. **Commitment Knowledge**:
   - The prover proves that the note commitment is mathematically valid: $$C = \text{Keccak256}(\text{asset} \mathbin{\Vert} v \mathbin{\Vert} \text{recipientPubkey} \mathbin{\Vert} \text{blinding})$$
4. **Range & Conservation**:
   - For all output notes $j \in [1, n]$: $$v_{\text{out}}^{(j)} \in [0, 2^{64}-1]$$
   - Conservation: $$\sum_{i=1}^m v_{\text{in}}^{(i)} = \sum_{j=1}^n v_{\text{out}}^{(j)} + f$$

4.3 On-Chain Verification (`IVerifier.sol`)

The EVM verifier contract evaluates pairing equations over the elliptic curve BN254.

```
interface IVerifier {
    function verifyProof(
        bytes calldata proof,
        uint256[] calldata pubSignals
    ) external view returns (bool);
}
```

Public Signals Layout:

The public signals array passed to `verifyProof` contains:

- `pubSignals[0]`: Historical Merkle Root (R)
- `pubSignals[1]`: Nullifier Hash ($\mathcal{N}$)
- `pubSignals[2]`: Output Commitment 1 ($C_{\text{out}}^{(1)}$)
- `pubSignals[3]`: Output Commitment 2 ($C_{\text{out}}^{(2)}$)
- `pubSignals[4]`: Public Amount (if unshielding, otherwise 0)
- `pubSignals[5]`: Public Recipient Address (if unshielding, otherwise 0)

Gas Benchmark Table:

Verification Operation	EVM Gas Cost	Execution Time
`ecAdd` (0x06 precompile)	150 gas	< 0.05 ms
`ecMul` (0x07 precompile)	6,000 gas	< 0.2 ms
`ecPairing` (0x08 precompile, 4 pairs)	185,000 gas	< 1.8 ms

Verification Operation	EVM Gas Cost	Execution Time
Total `verifyProof` Execution	**~210,000 gas**	**~2.1 ms**

4.4 Client-Side Prover Benchmarks (WASM & WebGPU)

To prevent centralized information leakage, **all zero-knowledge proofs are generated locally on the user's client device**. The private keys and blinding scalars never touch a remote server.

Seraph's proving engine compiles to WebAssembly (WASM) with SIMD acceleration and experimental WebGPU compute shaders:

Platform / Device	Proof Engine	Spend Proof Gen Time	Peak Memory
Apple M2 Max (Chrome / Desktop)	WebGPU Shaders	**1.24 s**	185 MB
Apple M2 Max (Node.js)	Native C++ / Rust	**0.88 s**	140 MB
Intel Core i7-12700K (Desktop)	WASM + SIMD	**2.15 s**	210 MB
iPhone 15 Pro (Safari / Mobile)	WebAssembly	**3.42 s**	195 MB
Samsung Galaxy S23 (Chrome)	WebAssembly	**4.10 s**	220 MB

4.5 Trusted Setup Transparency & Future STARK Upgrades

1. **Powers of Tau Ceremony**: Groth16 circuits rely on structured reference strings (SRS). Seraph utilizes the perpetual Powers of Tau ceremony (with over 1,000 international contributors). As long as one participant destroyed their toxic waste, the setup is secure.
2. **PLONK Universal Setup**: UltraPLONK implementations use KZG polynomial commitments over the same universal setup.
3. **STARK Upgrade Path**: Phase 4 of our research roadmap plans to integrate transparent, quantum-resistant STARKs (utilizing Rescue-Prime and Poseidon hashes) which eliminate trusted setups entirely.

Part II: Core Architecture & Networks

Base Layer settlement, Dual Private EVMs, Cross-EVM Bridges, Relayers, and MEV Shielding.

Chapter 5: Base Layer Protocol & State Machine

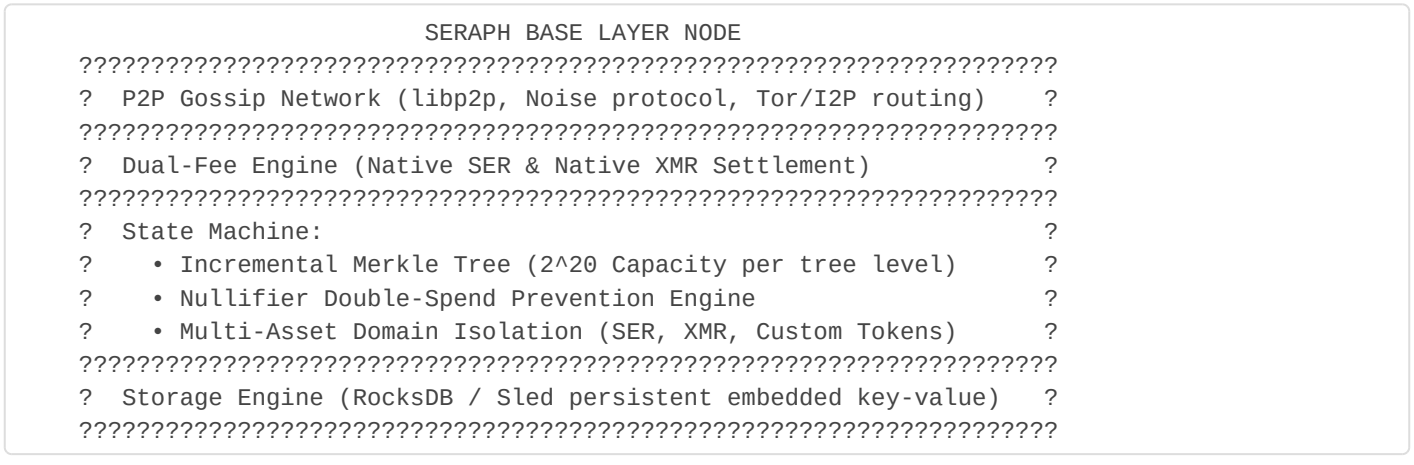
Chapter 5: Base Layer Protocol & State Machine

5.1 Protocol Architecture & Design Goals

The Seraph base layer is a custom high-performance blockchain node built in Rust (`seraph-base-layer`). It serves as the sovereign settlement substrate and shared anonymity pool for the entire Seraph ecosystem.

Unlike traditional EVM chains which record account balances directly in world state tries, the Seraph base layer maintains state purely through:

1. **The Global Commitment Tree**: An incremental Merkle tree accumulating all minted note commitments.
2. **The Nullifier Accumulator**: An append-only cryptographic registry of consumed nullifiers.



5.2 Block Structure & Consensus

Block Header Specification

Every block header on the Seraph base layer contains:

- `version`: Protocol version integer (32-bit).
- `previous_block_hash`: 32-byte cryptographic hash of parent block.
- `merkle_root`: Root of the transaction inclusion Merkle tree.
- `state_root`: Root of the active note commitment tree after processing all transactions.
- `nullifier_root`: Root of the newly consumed nullifiers batch.
- `timestamp`: 64-bit UTC UNIX timestamp.
- `height`: Monotonically increasing block index.
- `nonce`: Proof-of-work / validator signature depending on network phase.

Consensus Evolution

- **Phase 0 & 1 (Devnet & Testnet)**: Authority-validated sequencing with deterministic consensus for rapid testing and low latency.
- **Phase 2 & Beyond**: Transition to a hybrid Proof-of-Stake / RandomX-variant checkpointing engine, allowing native Monero miners to co-mine or validate Seraph blocks trustlessly.

5.3 Transactions and Enotes

In the Seraph base layer, transactions are strictly balance-conserving transitions consuming input notes and producing new output notes:

```
// Core Base-Layer Transaction Representation (base-layer/src/ledger.rs)
pub struct Transaction {
    pub inputs: Vec<[u8; 32]>,       // Spent note nullifiers
    pub outputs: Vec<Note>,          // New confidential notes
    pub fee_ser: u64,                // Fee paid in SER units
    pub fee_xmr: u64,                // Fee paid in atomic XMR units
    pub public_inputs: Vec<u8>,      // Merkle roots and public inputs
    pub proof: Vec<u8>,              // Cryptographic ZK spend proof
}
```

Transaction Validation Rules:

1. **Nullifier Uniqueness**: Every input nullifier must not currently exist in the ledger's `nullifiers` set: $$\forall \mathcal{N}_i \in \text{inputs}: \mathcal{N}_i \notin \text{Ledger.nullifiers}$$
2. **Atomic Nullifier Consumption**: All input nullifiers are inserted into `Ledger.nullifiers` within an atomic database transaction. If any nullifier fails, the entire transaction reverts.
3. **Commitment Insertion**: All output commitments $C_j = \text{output}_j.\text{commitment}()$ are inserted sequentially into the commitment tree.
4. **Conservation of Value**: The sum of inputs must equal the sum of outputs plus specified fees across each distinct asset class.

5.4 Dual-Fee Engine: Native SER and Native XMR

A signature innovation of the Seraph base layer is its native **dual-currency fee engine**. Transactions can pay inclusion fees in either:

1. **SER (Native Protocol Token)**: Supporting deflationary burning and treasury allocation.
2. **XMR (Native Monero)**: Bridged or natively integrated, allowing Monero holders to settle transactions directly without holding secondary gas tokens.

Fee Settlement Flow:

- When a user submits a transaction specifying `fee_xmr`, the transaction inputs consume atomic XMR enotes.
- Relayers or block proposers can receive fees in XMR directly into their private stealth addresses.
- Multi-asset isolation ensures that XMR and SER amounts never mix or cause rounding errors.

5.5 Multi-Asset Domain Isolation

Seraph avoids token cross-contamination by binding the `AssetId` directly into the leaf commitment hash:

```
// base-layer/src/assets.rs
pub enum AssetId {
    Seraph = 0,
    Xmr = 1,
    Custom(u32),
}

impl Note {
    pub fn commitment(&self) -> [u8; 32] {
        let mut hasher = Keccak256::new();
        hasher.update(self.asset.domain_tag());
        hasher.update(self.amount.to_be_bytes());
        hasher.update(self.recipient_pubkey);
        hasher.update(self.blinding);
        hasher.finalize().into()
    }
}
```

Even if an attacker crafts an XMR note with identical amount, recipient key, and blinding scalar as a SER note, the resulting commitments and nullifiers are cryptographically isolated.

5.6 Networking & P2P Topology

The networking stack scales through three distinct milestones:

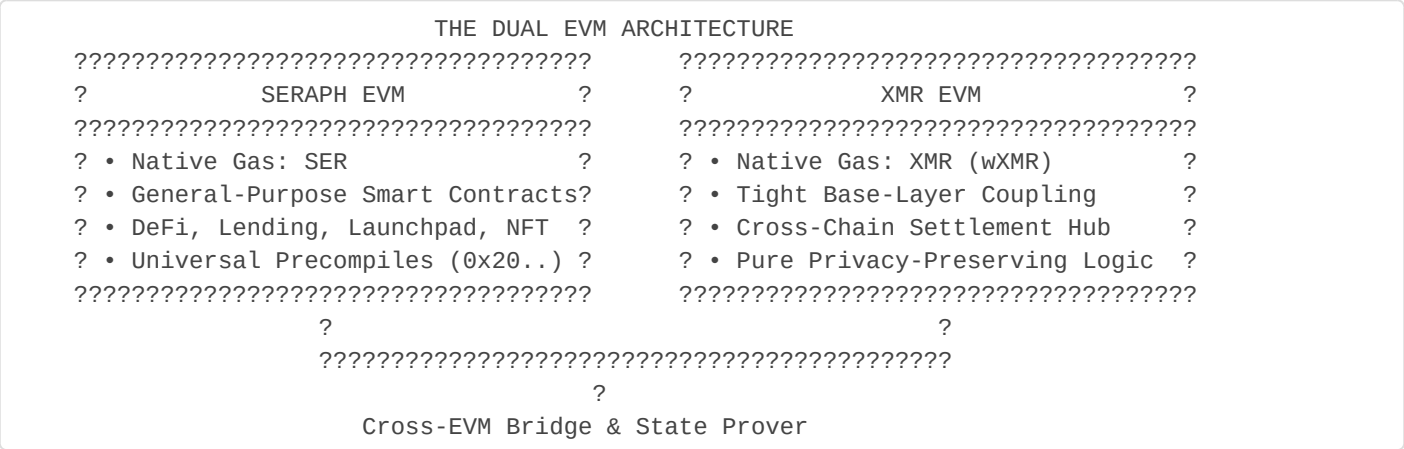
1. **Phase 0 (In-Process PoC)**: Validated in Rust via multi-actor test harnesses (`tests/adversarial_pentest.rs`) confirming state transitions, mempool double-spend rejections, and ring buffer evictions.
2. **Phase 1 (Single-Node Devnet)**: Local RPC node exposing JSON-RPC over HTTP and WebSocket, integrated with the `@seraph-protocol/sdk`.
3. **Phase 2 (Multi-Node Testnet)**: Fully decentralized libp2p gossip mesh with:
   - Dandelion++ transaction dissemination to prevent IP address triangulation.
   - Built-in Tor and I2P daemon proxies for complete network-layer anonymity.

Chapter 6: Dual Private EVMs

Chapter 6: Dual Private EVMs

6.1 Architectural Overview

Seraph deploys two specialized Ethereum Virtual Machines designed to provide programmable privacy without sacrificing developer tooling:



6.2 Seraph EVM: General-Purpose Confidential Execution

The **Seraph EVM** is an optimized EVM runtime where:

- **Base Fee Token**: Paid natively in SER tokens.
- **Contract Environment**: 100% bytecode compatible with EVM bytecode (Cancun/Prague), allowing existing Solidity, Vyper, and Yul contracts to deploy seamlessly.
- **Confidential Extensions**: Developers can import Seraph privacy libraries to shield variables, store encrypted state in storage slots, and authenticate callers via zero-knowledge proofs.

6.3 XMR EVM: Native XMR Settlement

The **XMR EVM** is a purpose-built sister EVM where:

- **Gas Settlement in XMR**: Gas fees are calculated in gas units but deducted from user balances in atomic XMR (piconeros) or bridged `wXMR`.
- **Base-Layer Synchronization**: The state of the XMR EVM is closely coupled to the base layer's Monero anonymity pool.
- **Cross-EVM Interoperability**: Contracts on the XMR EVM can trigger cross-contract calls into Seraph EVM via the Cross-EVM Bridge.

6.4 Custom Privacy Precompiles

Both EVM runtimes introduce specialized cryptographic precompiles located in the reserved precompile address range (`0x20` to `0x25`):

Address	Precompile Name	Description	Gas Cost
`0x20`	`PEDERSEN_COMMIT`	Computes $C = vH + rG$ over BN254	750 gas
`0x21`	`POSEIDON_HASH`	High-throughput algebraic hash for ZK circuits	300 gas
`0x22`	`STEALTH_DERIVE`	DKSAP derivation $P = \mathcal{H}_s(rA)G + B$	1,200 gas
`0x23`	`BULLETPROOF_VERIFY`	Verifies range proofs $v \in [0, 2^{64}-1]$	12,000 gas
`0x24`	`MERKLE_INSERT`	O(1) incremental Merkle tree insertion	2,500 gas
`0x25`	`NULLIFIER_CHECK`	Checks and consumes nullifiers in state	800 gas

These precompiles reduce the computational cost of zero-knowledge privacy operations by up to 90% compared to equivalent pure-Solidity implementations.

6.5 Confidential Contract State & Encrypted Variables

Traditional EVMs make all contract storage slots publicly readable by anyone running a node. Seraph introduces **Confidential Storage Slots**:

```
// Example: Confidential Storage in Solidity
contract PrivateBalanceVault {
    // Encrypted mapping: slot contains encrypted ciphertext + commitment
    mapping(address => bytes32) private encryptedBalances;

    // Homomorphic update via precompile
    function transferConfidential(
        bytes32 commitmentIn,
        bytes32 commitmentOut,
        bytes calldata zkProof
```

```
    ) external {
        require(IVerifier(0x23).verifyProof(zkProof, ...), "Invalid proof");
        // State updated without exposing raw balance
    }
}
```

- Data stored in confidential slots is encrypted with a contract-level ephemeral key.
- State modifications require valid zero-knowledge proofs of authorization.
- Public observers see state transitions occur, but cannot decode the underlying quantities.

6.6 Tooling & Developer Compatibility

Seraph maintains strict compatibility with standard Web3 development tools:

- **Hardhat & Foundry**: Compile and test using standard `forge build` and `forge test` without custom compiler forks.
- **Ethers.js & Viem**: Standard JSON-RPC interface support (`eth_call`, `eth_sendRawTransaction`, `eth_getLogs`).
- **MetaMask & Web3 Wallets**: Connect directly by adding RPC URL and Chain ID.
- **OpenZeppelin Contracts**: Inherit directly from standard OpenZeppelin v5 libraries (`ERC20`, `ReentrancyGuard`, `Ownable`).

Chapter 7: Bridges & Cross-Chain Architecture

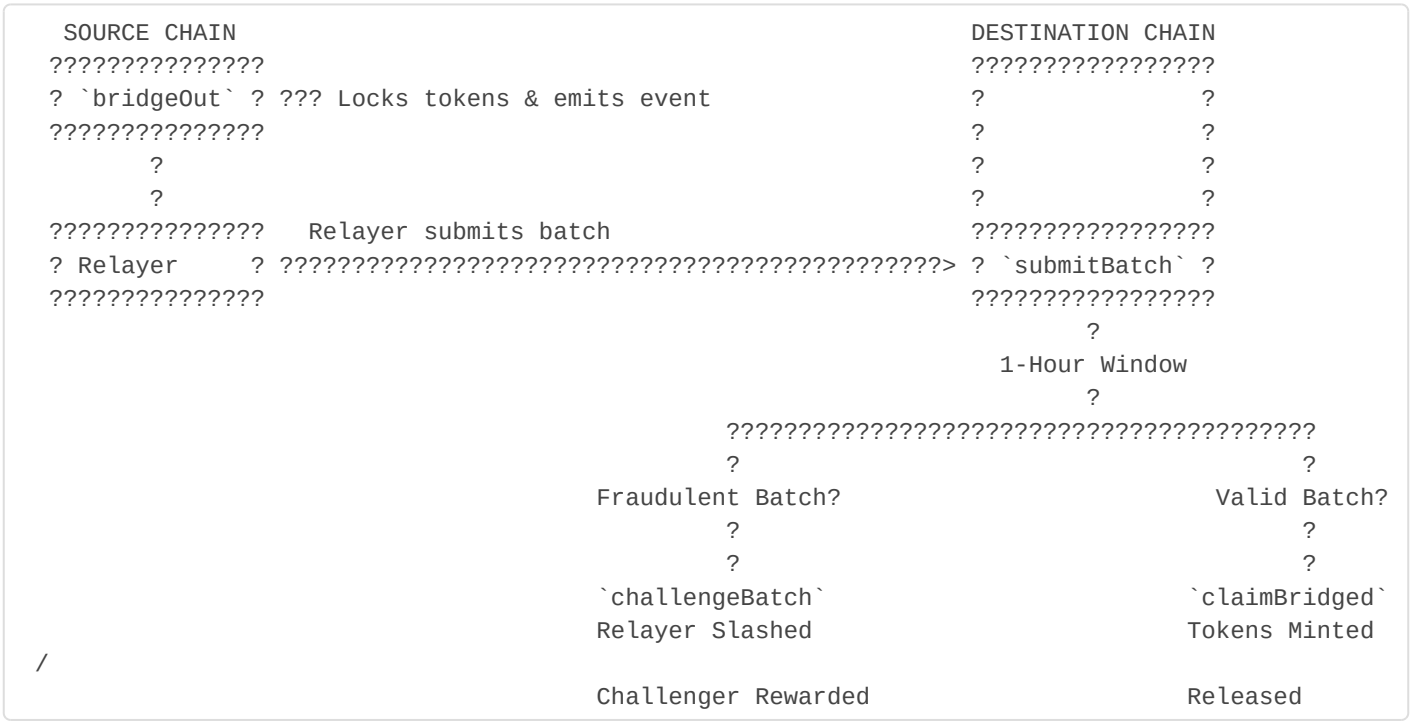
Chapter 7: Bridges & Cross-Chain Architecture

7.1 Cross-Chain Design Principles

The Seraph cross-chain architecture connects three heterogeneous execution environments:

1. **The Sovereign Seraph Base Layer** (Rust settlement & shared anonymity pool).
2. **Seraph EVM** (General-purpose EVM, gas in SER).
3. **XMR EVM** (Specialized EVM, gas in XMR/wXMR).

Rather than relying on trusted multisig federation bridges, Seraph implements an **optimistic, fraud-proof verified, nullifier-tagged bridge** (`CrossEVMBridge.sol`).



7.2 The Lock, Prove & Mint Mechanism

Cross-chain asset transfers follow a strict three-stage protocol:

Step 1: Lock on Source Chain

A user invokes `bridgeOut` on the source bridge:

```
function bridgeOut(
    address recipient,
    uint256 amount,
    bytes32 destChainId
) external nonReentrant returns (uint256 nonce);
```

- Tokens are locked into the bridge contract.
- A unique `outboundNonce` is assigned.
- An event `TokensLocked(nonce, sender, recipient, amount, destChainId)` is emitted.

Step 2: Optimistic Batch Submission

A bonded relayer packages outbound bridge transactions into a Merkle state root and submits it to the destination bridge:

```
function submitStateBatch(
    bytes32 sourceChainId,
    bytes32 stateRoot
) external whenNotPaused returns (uint256 batchId);
```

- Requires a minimum relayer bond of **1.0 ETH** (`MIN_RELAYER_BOND = 1 ether`).
- Timestamp is recorded, opening the **1-hour challenge window** (`CHALLENGE_WINDOW = 1 hours`).

Step 3: Withdrawal Claim

After the 1-hour window expires without challenge, the recipient calls `claimBridged`:

```
function claimBridged(
    uint256 batchId,
    bytes32 nullifier,
    address recipient,
    uint256 amount,
    bytes32[] calldata merkleProof
) external nonReentrant whenNotPaused;
```

- The bridge verifies the leaf: $$\text{leaf} = \text{Keccak256}(\text{nullifier} \mathbin{\Vert} \text{recipient} \mathbin{\Vert} \text{amount} \mathbin{\Vert} \text{localChainId})$$
- The bridge checks `merkleProof` against `batches[batchId].stateRoot`.
- The nullifier is recorded in `processedNullifiers[nullifier] = true`, preventing replay attacks.
- Tokens are unlocked or minted to the recipient.

7.3 Fraud Proofs, Bonds & Slashing

The bridge security model is economically protected through bonded collateral:

1. **Bond Requirements**: Any relayer wishing to submit batches must bond at least 1.0 ETH via `bondRelayer()`.
2. **Unbonding Locks**: When a relayer calls `unbondRelayer(amount)`, the contract verifies that no batches submitted by that relayer are currently inside the active 1-hour challenge window.
3. **Slashing Fraudulent Batches**: If a relayer submits an invalid root containing fictitious deposits, the protocol Guardian (or any automated watcher) invokes:

   ```
   function challengeBatch(uint256 batchId, address challenger) external nonReentrant;
   ```

   - The batch is immediately marked `challenged = true`, permanently preventing any token claims from that batch.
   - The relayer's entire bonded collateral is slashed to 0.
   - Slashed ETH is paid directly to the challenger as an economic bounty.

7.4 Emergency Pause Circuit Breakers

To protect user funds against zero-day vulnerabilities or upstream chain reorganizations, the bridge incorporates an emergency pause mechanism:

```
function setPaused(bool _paused) external {
    if (msg.sender != guardian) revert Unauthorized();
    paused = _paused;
    if (_paused) emit Paused(msg.sender);
    else emit Unpaused(msg.sender);
}
```

When paused:

- `bridgeOut()` reverts with `BridgePaused()`.
- `submitStateBatch()` reverts with `BridgePaused()`.
- `claimBridged()` reverts with `BridgePaused()`.
- Relayer bonds remain safely locked in escrow until the protocol guardian completes security review and unpauses.

7.5 Bridged XMR Representation (`wXMR`)

To represent native Monero on EVM layers:

- Monero locked in base-layer stealth multisig vaults is mirrored as an ERC-20 compliant `wXMR` token on Seraph EVM and XMR EVM.
- `wXMR` conforms to standard 18-decimal precision.
- Can be utilized across DeFi lending markets, automated liquidity pools, and gas payments via `GasRelayer`.

Chapter 8: Relayers & Gas Abstraction

Chapter 8: Relayers & Gas Abstraction

8.1 The Gas Dilemma in Confidential Systems

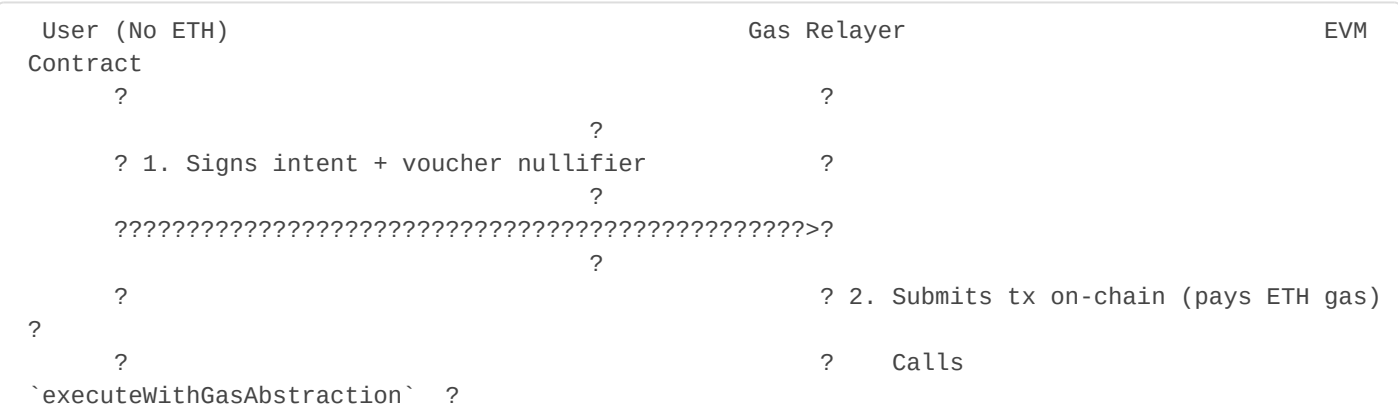
In traditional Web3 applications, interacting with a blockchain requires the user to hold native gas tokens (such as ETH) in their public wallet. This creates a severe privacy leak:

- Even if a user receives funds anonymously into a private stealth address, they cannot move those funds without funding that stealth address with gas.
- The gas funding transaction (originating from a CEX or public wallet) immediately establishes a graph-analysis link to the private address, completely destroying privacy.

Seraph eliminates this vulnerability through **account-abstracted gas relayers** (`GasRelayer.sol`).

8.2 The GasRelayer Architecture

The `GasRelayer` contract allows users to execute arbitrary EVM transactions without holding any native ETH in their executing wallet. Instead, transaction fees are settled using **pre-funded gas vouchers** in SER or XMR:



```
      ?                                                    ? ??????????????????????????????????????
>?
      ?                                                    ?
                                       ? 3. Executes user payload
      ?                                                    ? <
????????????????????????????????????????
      ?                                                    ? 4. Reimburses relayer in SER or
XMR   ?
      ?                                                    ?    from voucher pool
balance          ?
```

8.3 Voucher Funding & Nullifier Settlement

1. Funding a Gas Voucher

A user (or onboarding sponsor) deposits fee tokens into the voucher pool:

```
function fundGasVoucher(
    address feeToken,
    uint256 amount,
    bytes32 voucherCommitment
) external nonReentrant;
```

- `feeToken`: SER or `wXMR`.
- `voucherCommitment`: A cryptographic hash committing to an unspent secret scalar.
- Tokens are held in the contract's pooled liquidity escrow.

2. Gas-Abstracted Execution

When the user desires to execute a call:

```
function executeWithGasAbstraction(
    bytes32 voucherNullifier,
    address feeToken,
    uint256 feeAmount,
    address target,
    bytes calldata data
) external nonReentrant returns (bytes memory result);
```

- The relayer broadcasts the transaction and pays the native gas in ETH.
- The contract verifies that `voucherNullifier` has not been used (`!usedVouchers[voucherNullifier]`).
- The contract verifies that `voucherPoolBalance[feeToken] >= feeAmount`.
- The contract marks `usedVouchers[voucherNullifier] = true`.
- The contract transfers `feeAmount` of `feeToken` directly to the relayer as compensation.
- The contract executes `target.call(data)` on behalf of the user.

8.4 Defensive Security & Target Filtering

To prevent malicious users from crafting exploit calls that drain protocol assets or hijack the relayer, `GasRelayer.sol` implements strict target validation:

```
error ForbiddenTarget();

if (
    target == address(0) ||
    target == address(this) ||
    target == address(serToken) ||
    target == address(wxmrToken)
) revert ForbiddenTarget();
```

Attack Mitigation:

- An attacker cannot supply `target = address(serToken)` with `data = transfer(...)` to drain tokens held by the relayer pool.
- An attacker cannot call internal admin functions on `GasRelayer` itself.
- Reentrancy locks (`nonReentrant`) prevent recursive callback drains.

8.5 Gas Rate Oracles & Fee Adjustment

The conversion rate between native ETH gas and fee tokens (SER / XMR) is dynamically adjusted by governance:

```
function setTokenGasRate(address feeToken, uint256 rate) external onlyOwner;
```

This ensures that relayers remain profitable across market volatility while users pay minimal overhead (typically < 3% over raw gas costs).

Chapter 9: Sequencer Architecture & MEV Shielding

Chapter 9: Sequencer Architecture & MEV Shielding

9.1 The MEV Vulnerability in Transparent Blockchains

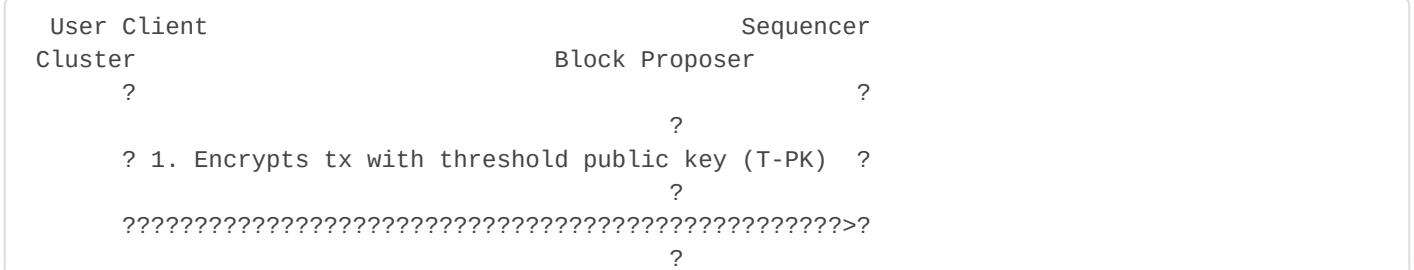
On public smart contract networks, transactions wait in a transparent memory pool (mempool) prior to block inclusion. This allows predatory searchers and maximal extractable value (MEV) bots to:

1. **Front-Run**: Detect profitable DEX trades and insert their own transaction ahead by paying higher priority gas.
2. **Sandwich Attack**: Enclose a user's swap between a buy and sell transaction, intentionally extracting slippage from ordinary users.
3. **Censorship & Liquidation Sniping**: Reorder transactions to trigger liquidations or censor privacy transactions.

In a privacy protocol, MEV poses an existential threat: if transaction ordering or metadata leaks before execution, privacy guarantees can be severely compromised.

9.2 Encrypted Mempool Architecture

Seraph deploys an **Encrypted Mempool with Threshold Cryptography** to eliminate front-running and toxic MEV:





Protocol Steps:

1. **Client-Side Encryption**: When a user constructs a transaction, the payload (recipient, function selector, parameters, value) is encrypted using a distributed **Threshold Public Key (PK_{thresh})**: $$C_{\text{tx}} = \text{Encrypt}(PK_{\text{thresh}}, \text{tx})$$ Only the gas bid and sender's zero-knowledge validity proof are visible to relayers.
2. **Order Commitment**: Sequencer nodes receive C_{tx} and commit to an immutable ordered sequence: $$H_{\text{batch}} = \text{Keccak256}(C_1 \mathbin{\Vert} C_2 \mathbin{\Vert} \dots \mathbin{\Vert} C_k)$$ Once H_{batch} is posted to the base layer, the transaction order is mathematically locked and cannot be altered.
3. **Threshold Decryption**: A distributed committee of n validator nodes each release an ephemeral decryption key share. Once t of n shares (e.g., 5-of-7 quorum) are broadcast, the full batch is decrypted simultaneously.
4. **Deterministic Execution**: The decrypted transactions are executed in the exact pre-committed order. Because searchers cannot see trade parameters before the order is fixed, front-running and sandwiching are mathematically impossible.

9.3 Protection Against Toxic MEV

Attack Vector	Vulnerability on Public EVMs	Mitigation on Seraph Dual EVMs
DEX Sandwich Attacks	Searchers front-run & back-run slippage	**Impossible**: Swap amounts and paths are hidden inside threshold ciphertext until execution order is locked.
Liquidation Front-Running	Bots compete for liquidation priority gas	**Fair Ordering**: First-seen encrypted batches are executed sequentially without priority auction bidding.

Attack Vector	Vulnerability on Public EVMs	Mitigation on Seraph Dual EVMs
Statistical Arbitrage	Bots front-run oracle price updates	**Time-locked Batches**: Price updates and user transactions settle within atomic execution blocks.
Censorship	Validators selectively exclude privacy txs	**Zero Metadata**: Validators cannot distinguish between privacy pool deposits, transfers, and standard swaps.

9.4 Decentralized Sequencer Rotation & Fault Recovery

To prevent single-point-of-failure centralization, Seraph implements a **Decentralized Sequencer Set (DSS)**:

1. **Epoch-Based Rotation**: A new primary sequencer is elected deterministically every 100 blocks based on bonded SER stake.
2. **Slashing for Non-Liveness**: If a sequencer fails to submit a batch within the 2-second block window, the standby backup sequencer immediately takes over, and the inactive sequencer's stake is slashed.
3. **Equivocation Slashing**: If a sequencer signs two conflicting batch orders for the same block height, an automated fraud proof slashes 100% of their bonded stake.

Part III: Applications & Tokenomics

Wallets, First-Party Privacy Modules (xUSD CDP, AI Usage Tickets, Stealth Invoices), DEX/CEX integrations, SER Tokenomics, Presale, and Creator Launchpad.

Chapter 10: Wallets, Scanning & Key Recovery

Chapter 10: Wallets, Scanning & Key Recovery

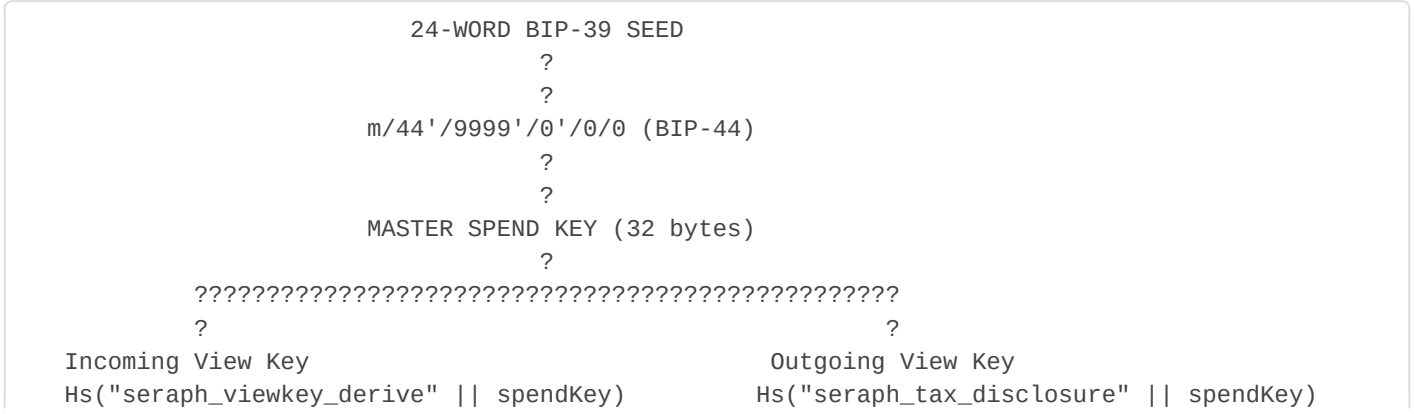
10.1 The Official Seraph Wallet Ecosystem

The Seraph wallet ecosystem provides non-custodial key management and privacy execution across desktop, mobile, and web interfaces:

- **Desktop Wallet (macOS, Linux, Windows)**: Full-featured native application with local zero-knowledge prover acceleration and optional integrated full-node synchronization.
- **Web Wallet & Browser Extension**: Lightweight client integrated directly into modern browsers, utilizing WASM and WebGPU shaders for client-side proof generation.
- **Mobile Wallet (iOS, Android)**: Fast, touch-optimized wallet with biometric key protection and delegated scan-key processing.

10.2 Key Hierarchy & Derivation

Seraph derives all cryptographic keys deterministically from a single standard 24-word BIP-39 mnemonic seed phrase:



```
(Scans & Decrypts Incoming Notes)          (Signs Verifiable Audit Packages)
         ?                                          ?
         ?                                          ?
Stealth Public Key                         Compliance Signature
Hs("seraph_pubkey_derive" || spendKey)     Tax / Audit Verification
```

10.3 Client-Side Note Scanning (`scanNotesForOwner`)

Because note commitments on the blockchain are cryptographically blinded, a wallet must scan new ledger blocks to identify which notes belong to the user.

In Seraph, this is accomplished via **trial decryption** using the wallet's **Incoming View Key**:

```
// Production Note Scanning Engine (sdk/typescript/src/index.ts)
const scannedNotes = scanNotesForOwner(encryptedNotes, incomingViewKey);
```

The Magic Tag & Decryption Layout:

Every note memo payload is encoded with a 4-byte magic identifier (`NOTE_MAGIC = "SERP"` = `0x53455250`): $$\text{Memo Layout} = [\text{4 bytes "SERP"}] \mathbin{\Vert} [\text{8 bytes Amount BE}] \mathbin{\Vert} [\text{1 byte AssetId}] \mathbin{\Vert} [\text{UTF-8 Memo}]$$

1. The wallet iterates through new note commitments.
2. For each note, the wallet decrypts the ciphertext using its `incomingViewKey` and the note's public `blinding`: $$\text{decrypted} = \text{xorViewKeyMemo}(\mathcal{K}_{\text{in}}, \text{blinding}, \text{encryptedMemo})$$
3. The wallet inspects the first 4 bytes:
   - If `decrypted[0..3] == 0x53455250`, the note **definitely belongs to this wallet**!
   - If the magic does not match, the note belongs to another user and is instantly discarded.
4. For matched notes, the wallet decodes the exact spendable balance, the asset type (`AssetId.Seraph` or `AssetId.Xmr`), and the secret transaction memo.

10.4 Verifiable Tax & Compliance Disclosures

Privacy should not require users to violate financial reporting laws. Seraph enables users to generate mathematical, verifiable compliance disclosure packages signed by their **Outgoing View Key**:

```
// Generate Tax Package (sdk/typescript/src/index.ts)
const disclosure = generateComplianceDisclosure(spentRecords, outgoingViewKey);
```

Disclosure Package Structure:

- `disclosureSummary`: JSON payload enumerating:
  - Consumed nullifiers ($\mathcal{N}_i$).
  - Exact unshielded amounts.
  - Destination public addresses.
  - Timestamps and on-chain transaction hashes.
- `signatureTag`: 32-byte cryptographic HMAC tag: $$\text{signatureTag} = \text{SHA256Tag}(\text{"seraph_tax_disclosure_v1"}, \mathcal{K}_{\text{out}}, \text{summaryBytes})$$

An auditor or tax accountant possessing the user's `outgoingViewKey` can verify the signature and prove that all reported transactions are authentic and comprehensive, without being given spending authority.

10.5 Encrypted Wallet Backups & Paper Recovery

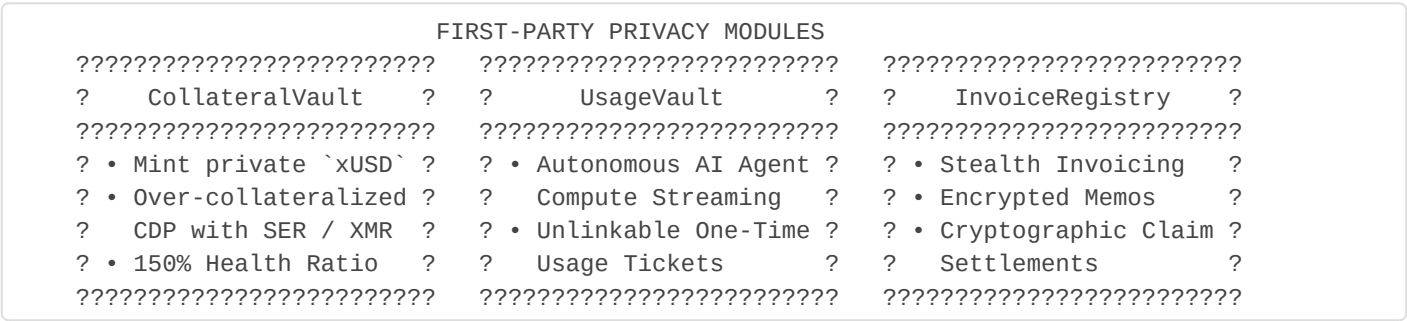
1. **Standard Seed Phrase**: The 24-word seed phrase is sufficient to recover 100% of all past, present, and future funds and transaction history.
2. **Encrypted JSON Vault**: Desktop and web wallets allow exporting an AES-GCM-256 encrypted backup file containing cached Merkle paths, note commitments, and local address labels for instantaneous wallet restore without requiring a full chain rescan.
3. **Emergency Paper Key**: A printable cold-storage certificate containing the master spend key and incoming view key in high-density QR code format.

Chapter 11: First-Party Privacy Modules

Chapter 11: First-Party Privacy Modules

11.1 The Ecosystem of First-Party Privacy Modules

Seraph extends beyond simple token transfers by providing an integrated suite of native, first-party privacy applications deployed directly on the confidential EVMs:



11.2 Private Stablecoins: `xUSD` & `CollateralVault`

Volatility in crypto assets makes everyday commerce difficult. The `CollateralVault` enables users to lock SER or XMR collateral and mint **`xUSD`**—a confidential, decentralized dollar stablecoin.

Mechanics & Invariants:

1. **Minimum Collateralization Ratio (150%)**: $$\text{Collateral Ratio} = \frac{\text{Collateral} \times \text{Price}_{\textit{USD}}}{\textit{Debt}_{\text{xUSD}}} \ge 150\%$$
2. **Deposit & Mint**:

```
function depositAndMint(
    uint256 collateralAmount,
    uint256 mintAmount
) external nonReentrant;
```

Users deposit collateral and mint synthetic `xUSD` directly into their private balances.

3. **Burn & Withdraw**:

```
function burnAndWithdraw(
    uint256 burnAmount,
    uint256 withdrawCollateral
) external nonReentrant;
```

Users burn `xUSD` to unlock their underlying collateral, maintaining the 150% ratio.

4. **Confidential Circulation**: Once minted, `xUSD` can be shielded and transferred through the `PrivacyPool` with zero public record of sender, recipient, or balance.

11.3 Autonomous AI Agent Billing: `UsageVault`

As autonomous AI agents, machine learning inference nodes, and cloud compute clusters transact on-chain, they require high-frequency, low-latency, unlinkable micro-payments without exposing corporate infrastructure budgets.

The `UsageVault` implements **cryptographic one-time usage tickets**:

```
 Client / AI Agent                                    UsageVault
Contract                              AI Compute Provider
        ?                                                    ?
                                                ?
        ? 1. Deposits credits (`depositCredits`)             ?
                                                ?
        ?    Commitment = H(amount, secretPreimage)          ?
                                                ?
        ?????????????????????????????????????????????????>?
                                                ?
        ?                                                    ?
                                                ?
        ? 2. Sends one-time ticket off-chain to Provider    ?
                                                ?
        ?    (ticketNullifier = H(secretPreimage), amount)   ?
                                                ?
        ?
????????????????????????????????????????????????????????????????????????????????????????????????????
???>?
        ?                                                    ?
                                                ?
        ?                                                    ? 3. Redeems ticket
(`redeemUsageTicket`)       ?
        ?                                                    ?    Verifies H(amount, secret) ==
commitment   ?
        ?                                                    ? <
```

```
????????????????????????????????????????????
        ?                                                    ?
                                                ?
        ?                                                    ? 4. Withdraws provider
revenue                 ?
        ?                                                    ?
(`claimEarnings`)                           ?
```

SDK Integration:

```
import { createCreditDeposit, UsageVaultClient } from "@seraph-protocol/sdk";

// 1. Create unlinkable credit ticket
const secretPreimage = randomBytes32();
const ticket = createCreditDeposit(100n * 10n**18n, secretPreimage);

// 2. Deposit into vault
const client = new UsageVaultClient(vaultAddress);
const tx = client.encodeDeposit(ticket.commitment, 100n * 10n**18n);
```

11.4 Stealth Invoicing & Encrypted Messaging: `InvoiceRegistry`

The `InvoiceRegistry` contract solves privacy for merchant invoicing and peer-to-peer business transactions:

1. Invoice Creation

```
function createInvoice(
    bytes32 invoiceId,
    bytes32 stealthRecipient,
    address claimAddr,
    address token,
    uint256 amount,
    bytes calldata encryptedMemo
) external nonReentrant;
```

- The merchant publishes an invoice bound to a stealth destination address.
- The memo (specifying goods, order ID, or service description) is encrypted under the buyer's view key.

2. Settle & Nullify

```
function settleInvoice(
    bytes32 invoiceId,
    bytes32 paymentNullifier
) external nonReentrant;
```

- When the payment is executed, the `paymentNullifier` is permanently recorded.
- Replaying payments across different invoices is strictly prevented (`NullifierAlreadyUsed`).

3. End-to-End Encrypted Messaging

Users can broadcast encrypted messages to any recipient's public key:

```
function sendEncryptedMessage(
    bytes32 recipientPubkey,
    bytes32 ephemeralPubkey,
    bytes calldata ciphertext
) external;
```

This enables native on-chain communication for trade negotiations, OTC settlement confirmations, and payment notices.

Chapter 12: Ecosystem, DEX & CEX Integrations

Chapter 12: Ecosystem, DEX & CEX Integrations

12.1 Overview: Integrating Privacy with Global Liquidity

Decentralized finance (DeFi) liquidity and centralized exchange (CEX) listings require standard, predictable ERC-20 token interactions. However, protocols featuring transfer taxes or non-standard balance logic frequently experience routing failures, slippage mismatches, or unexpected fees during multi-hop DEX swaps.

Seraph solves this through a dedicated **Tax Exemption Registry** and standardized integration interfaces designed specifically for automated market makers (AMMs) and institutional custodians.

12.2 Decentralized Exchanges (Uniswap v3 & Aerodrome)

The Multi-Hop Double-Tax Problem

On a standard DEX, a swap from SER to USDC often routes through an intermediary pool: $$\text{User} \xrightarrow{\text{Swap}} \text{Pool}*{\text{SER/WETH}} \xrightarrow{\text{Internal Transfer}} \text{Router} \xrightarrow{\text{Swap}} \text{Pool}*{\text{WETH/USDC}} \xrightarrow{} \text{User}$$ If a 1.00% sell tax is assessed on every hop, the user would be taxed twice (or the DEX router would fail due to received amount mismatches).

The Tax Exemption Mapping (`taxExempt`)

To ensure seamless integration with Uniswap v3, Aerodrome, Curve, and Balancer, `SERAPH.sol` includes a dynamic exemption mapping:

```
// contracts/core/SERAPH.sol
mapping(address => bool) public taxExempt;
event TaxExemptionUpdated(address indexed account, bool isExempt);

function setTaxExempt(address account, bool isExempt) external onlyTreasury {
    taxExempt[account] = isExempt;
    emit TaxExemptionUpdated(account, isExempt);
}
```

In the internal `_update` transfer hook:

```
bool isSell = from != address(0) && to != address(0) && from != address(this) && to !=
address(this)
    && from != presale && to != presale && from != treasury && to != treasury
    && from != privacyPool && to != privacyPool
    && !taxExempt[from] && !taxExempt[to];
```

Whitelisting DEX Contracts:

1. When a new Uniswap v3 pool or Aerodrome gauge is deployed, the Seraph Treasury (governed by the Protocol DAO) invokes:
   ```
   seraphToken.setTaxExempt(uniswapV3PoolAddress, true);
   seraphToken.setTaxExempt(universalRouterAddress, true);
   ```
2. Once whitelisted, transfers between the router, pools, and liquidity providers execute with **0% tax**.
3. Sells initiated by end users on public markets continue to contribute to the deflationary burn and protocol treasury.

12.3 Centralized Exchanges (CEX) Custody & Operations

Centralized exchanges (such as Coinbase, Binance, OKX, and Kraken) utilize hot-and-cold wallet architectures where funds are swept internally between omnibus deposit addresses, cold storage vaults, and hot withdrawal wallets.

Custody Whitelisting Workflow:

1. Prior to listing, the exchange provides their omnibus hot/cold wallet addresses to the Seraph Treasury.
2. The Treasury adds these custody addresses to `taxExempt`:
   ```
   seraphToken.setTaxExempt(cexOmnibusWallet, true);
   ```
3. **Benefits**:
   - Internal exchange balance sweeps incur **0% tax**.
   - User deposits and withdrawals match exact balances to the single wei.
   - Zero reconciliation discrepancy for exchange accounting databases.

12.4 Autonomous AI Agent Payment SDK Integration

Autonomous AI agents transacting across decentralized networks can integrate `@seraph-protocol/sdk` with zero external dependencies:

```
import {
  deriveWalletKeys,
  computeNoteCommitment,
  UsageVaultClient,
  createCreditDeposit,
  AssetId,
} from "@seraph-protocol/sdk";

// Initialize autonomous agent wallet
const agentSpendKey = Buffer.from(process.env.AGENT_SPEND_KEY!, "hex");
const keys = deriveWalletKeys(agentSpendKey);

// Prepare usage credit deposit for compute streaming
const computeCost = 10n * 10n**18n; // 10 SER
const secretPreimage = crypto.randomBytes(32);
const ticket = createCreditDeposit(computeCost, secretPreimage);

// Submit on-chain via client
const vaultClient = new UsageVaultClient(USAGE_VAULT_ADDRESS);
const tx = vaultClient.encodeDeposit(ticket.commitment, computeCost);
```

12.5 Standardized JSON-RPC & WebSocket Streaming

Seraph nodes expose standard JSON-RPC 2.0 endpoints for ecosystem tooling:

Custom Privacy RPC Methods:

- `seraph_getLastRoot()`: Returns the latest Merkle root of the active commitment tree.
- `seraph_isKnownRoot(root)`: Verifies if a given 32-byte root exists within the 100-root ring buffer.
- `seraph_getNotesByRange(startLeaf, endLeaf)`: Fetches encrypted note memo payloads for client-side scanning.
- `seraph_estimateGasVoucher(feeToken, gasLimit)`: Calculates the required SER or XMR fee voucher amount for gasless execution.

WebSocket Subscriptions:

Clients can listen to live on-chain privacy events via WebSocket:

```
const ws = new WebSocket("wss://rpc.seraphprotocol.tech/ws");
ws.send(JSON.stringify({
  jsonrpc: "2.0",
  id: 1,
  method: "eth_subscribe",
  params: ["logs", { address: PRIVACY_POOL_ADDRESS }]
}));
```

This allows wallets to detect incoming payments in real time (< 1 second latency).

Chapter 13: SER Tokenomics, Supply & Burns

Chapter 13: SER Tokenomics, Supply & Burns

13.1 Fixed Total Supply & Hard Cap

The native utility and governance token of Seraph Protocol is **SER**. The token is governed by a strictly immutable supply cap enforced at the smart contract level:

$$\text{MAX_SUPPLY} = 100,000,000 \times 10^{18} \text{ wei } (100\text{ Million SER})$$

In `contracts/core/SERAPH.sol`:

```
uint256 public constant MAX_SUPPLY = 100_000_000 * 1e18;

function mint(address to, uint256 amount) external onlyMinter {
    require(totalSupply() + amount <= MAX_SUPPLY, "SeraphToken: max supply exceeded");
    _mint(to, amount);
}
```

No administrative role, multisig, or governance proposal can ever expand supply beyond 100M tokens.

13.2 Genesis Token Allocation

The 100,000,000 SER token allocation is structured to balance community participation, decentralized ecosystem bootstrapping, and long-term protocol security:

```
                        100M SER ALLOCATION
   ??????????????????????????????????????????????????????????????????
   ? [??????????????????????] Presale & Community Tiers   (30.0%)   ?
   ? [??????????????????]       Protocol Treasury & DAO     (25.0%)   ?
   ? [???????????????]          Liquidity & DEX Bootstrapping(20.0%)  ?
   ? [??????????]               Core Contributors & Team    (15.0%)   ?
   ? [??????]                   Ecosystem Grants & Research (10.0%)   ?
   ??????????????????????????????????????????????????????????????????
```

Category	Allocation	Token Amount	Vesting Schedule
Presale (Strategic, Private, Public)	30.0%	30,000,000 SER	Strategic: 25% TGE, 14d cliff, 12m linear Private: 20% TGE, 14d cliff, 12m linear Public: 100% TGE unlock
Protocol Treasury & DAO	25.0%	25,000,000 SER	Multi-year DAO governance release
Liquidity & DEX Bootstrapping	20.0%	20,000,000 SER	100% unlocked at TGE for AMM depth
Core Contributors & Team	15.0%	15,000,000 SER	12-month cliff, 36-month linear vesting
Ecosystem Grants & Research	10.0%	10,000,000 SER	Milestone-gated via DAO proposals

---

13.3 Multi-Faceted Token Utility

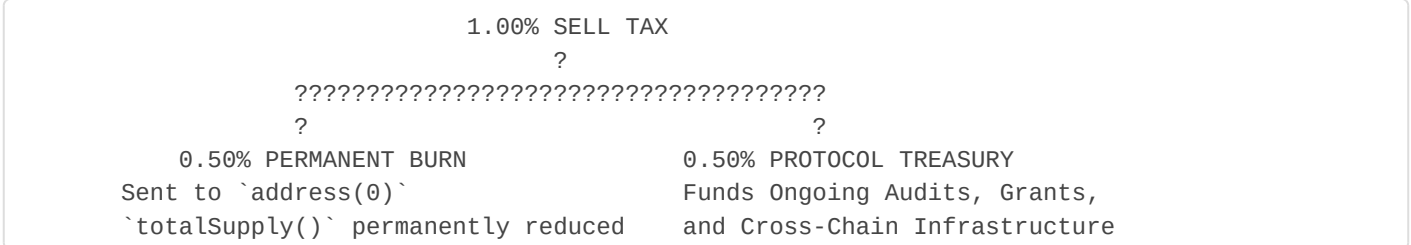
SER powers all economic activity across the Privacy Continuum:

1. **Gas Fee Settlement**: Primary gas asset for executing contracts and state transitions on Seraph EVM.
2. **Privacy Pool Collateral**: Staked or locked in `PrivacyPool` and `CollateralVault` to mint confidential synthetic `xUSD`.
3. **Decentralized Governance**: Token holders vote on protocol parameters, precompile additions, and treasury grants using anonymous zero-knowledge ballots.
4. **Relayer & Sequencer Staking**: Relayers bond SER to earn routing fees on the Cross-EVM Bridge and participate in block sequencing.

13.4 The Deflationary Sell Tax & Burn Engine

To ensure long-term value accrual and discourage predatory mercenary dumping, `SERAPH.sol` incorporates an automated sell-side fee mechanism:

- **Sell Tax Rate**: 1.00% (`SELL_TAX_BPS = 100`).
- **Burn Split**: 50% permanently burned (`BURN_BPS = 5000`).
- **Treasury Split**: 50% directed to the Protocol Treasury (`toTreasury = tax - burned`, official address: `0xa287FC2B8f89023eF1c82af09aee351cCdDa8a76`).



Invariant & Mathematical Proofs

Our test suite rigorously verifies two inviolable economic invariants across random fuzz actions (`PermitTaxAndPresaleStress.t.sol`):

1. **Supply Conservation**: $$\text{totalSupply}() + \text{totalBurned} == \text{totalMinted}$$
2. **Absolute Ceiling**: $$\text{totalSupply}() \le \text{MAX_SUPPLY}$$

Zero Tax on Shielding & Unshielding:

Transfers into and out of the `PrivacyPool` are strictly exempt from the sell tax:

- `shield()`: 0% Tax.
- `privateTransfer()`: 0% Tax.
- `unshield()`: 0% Tax. This guarantees that privacy preservation is never penalized with protocol friction.

Chapter 14: Presale Architecture & Vesting

Chapter 14: Presale Architecture & Vesting

14.1 Presale Philosophy & Structural Design

The Seraph presale (`Presale.sol`) is designed to distribute early protocol ownership transparently across three tiers while preventing post-TGE dumping through mathematical vesting schedules.

The presale contract is deployed and verified on Base Sepolia (`0x2c4156e4bFE0bE6A4e683C7Be521E1cC10B798bE`) and supports seamless multi-currency investments in **ETH**, **USDC**, and **USDT**.

14.2 Tier Specifications & Allocation Matrix

Tier ID	Tier Name	Token Price	Allocation Cap	TGE Unlock	Cliff Period	Linear Vesting	Access Gating
Tier 1	**Strategic**	**$0.02**	10,000,000 SER	**25.0%**	14 Days	12 Months	Whitelist / KYC Required
Tier 2	**Private**	**$0.03**	10,000,000 SER	**20.0%**	14 Days	12 Months	Whitelist / KYC Required
Tier 3	**Public**	**$0.04**	10,000,000 SER	**100.0%**	None (0 Days)	None (0 Months)	Open to All (No KYC)

Key Economic Properties:

- **Total Raised Potential**: $$\text{Strategic: } 10\text{M} \times $0.02 = $200,000$$ $$\text{Private: } 10\text{M} \times $0.03 = $300,000$$ $$\text{Public: } 10\text{M} \times $0.04 = $400,000$$ $$\text{Total Target Hard Cap} = $900,000$$
- **Presale Protocol Fee**: A nominal **0.50% fee** (`FEE_BPS = 50`) is deducted on purchases to cover deployment and security audit costs.

14.3 Multi-Currency Payment Engine

Investors can purchase allocations using their preferred asset:

1. **USDC (6 Decimals)**: `buyWithStablecoin(tierId, amount, usdc, referrer)`
2. **USDT (6 Decimals)**: `buyWithStablecoin(tierId, amount, usdt, referrer)`
3. **ETH (18 Decimals)**: `buyWithETH(tierId)` or direct wallet transfer via `receive()` fallback.

Dynamic ETH/USD Pricing:

The presale contract maintains an on-chain price parameter (`ethUsdPrice`) updated by the Treasury to mirror real-time market rates: $$\text{Tokens Purchased} = \frac{\text{ETH Amount} \times \text{ethUsdPrice}}{\text{Tier Price}}$$ Normalizing 6-decimal stablecoins to 18-decimal token precision is handled automatically with full integer precision and zero truncation loss.

14.4 Vesting Mathematics & Claim Lifecycle

For an investor with total allocation A in Tier 1 or Tier 2:

Phase 1: TGE Claim (`claimInitialAllocation`)

At Token Generation Event ($t \ge T_{\text{TGE}}$): $$\text{Initial Claimable} = A \times \frac{\text{UnlockBPS}}{10,000}$$

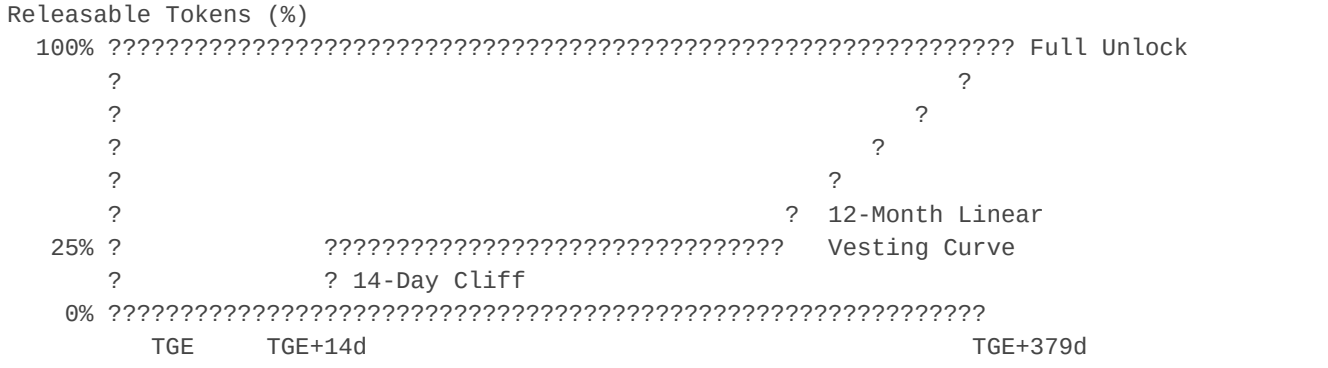
- Strategic investors claim **25%** immediately.
- Private investors claim **20%** immediately.
- Public investors claim **100%** immediately via `claimInitialAllocation()`.

Phase 2: The Cliff Period

For the first 14 days following TGE ($T_{\text{TGE}} \le t < T_{\text{TGE}} + 14 \text{ days}$): $$\text{Vested Amount} = 0$$ No additional tokens are releasable during the cliff.

Phase 3: Continuous Linear Vesting (`claimVested`)

Following the 14-day cliff ($t \ge T_{\text{TGE}} + 14 \text{ days}$): $$\text{Releasable}(t) = \left(A - \text{Initial} \right) \times \min\left(1, \frac{t - (T_{\text{TGE}} + 14 \text{ days})}{365 \text{ days}}\right) - \text{AlreadyReleased}$$



Zero-Dust Guarantee

Our test suite confirms that after $t \ge T_{\text{TGE}} + 379 \text{ days}$: $$\text{v.released} == \text{v.total} \quad (\text{Exact to the single wei})$$ No fractional wei is permanently stranded in the contract.

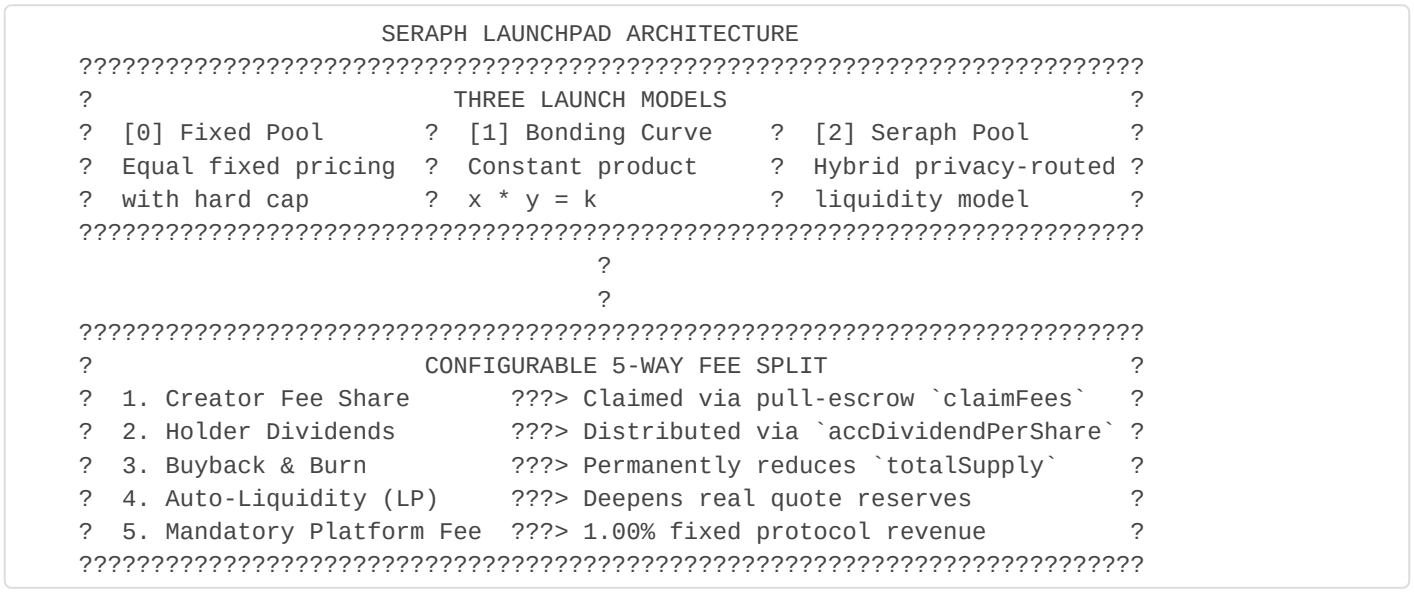
Chapter 15: Launchpad Mechanics & Creator Playbook

Chapter 15: Launchpad Mechanics & Creator Playbook

15.1 Overview: Fair Launches for the Privacy Era

The Seraph Launchpad (`Launchpad.sol`) enables creators, autonomous agents, and decentralized communities to launch privacy-native ERC-20 tokens with customizable economic flywheels, continuous bonding curves, and automated dividend distributions.

The contract is live on Base Sepolia at `0xe0349E4cC431329207e98C5205062776e058A846`.



15.2 The Three Launch Models

1. **Fixed Pool (`Model 0`)**:

- Fixed token price from launch to graduation.
- Ideal for predictable public token allocations and community airdrop claims.

2. **Bonding Curve (`Model 1`)**:
   - Continuous constant-product automated market maker ($x \cdot y = k$).
   - Token price rises monotonically on every buy and falls on every sell.
   - Phantom virtual liquidity (Q_{phantom}) provides initial price stability without requiring upfront creator capital.
3. **Seraph Pool (`Model 2`)**:
   - Hybrid bonding curve tightly integrated with the `PrivacyPool`.
   - Buyers can opt to shield received tokens directly into private notes in the same transaction.

15.3 The Fee Model, Anti-Spam Launch Fee & 6 UI Presets

The launchpad charges:

- A **flat anti-spam creation fee of 0.001 ETH** (~$2.50 USD), paid at token deployment and routed directly to the Seraph Treasury (`0xa287FC2B8f89023eF1c82af09aee351cCdDa8a76`) to prevent automated bot flood attacks.
- A **mandatory 1.00% Seraph platform fee** on every buy and sell (`PLATFORM_FEE_BPS = 100`).
- A **creator fee of up to 2.50%** (`buyFeeBps`, `sellFeeBps` ≤ 250).

Creators allocate their trading fee revenue across 5 destinations summing to 10,000 basis points (100%):

$$\text{creatorShare} + \text{buybackShare} + \text{dividendsShare} + \text{liquidityShare} + \text{platformShare} = 10,000 \text{ bps}$$

The 6 Creator UI Presets:

1. **Creator-Backed**: 100% of creator fees to creator wallet. Best for artists, builders, and direct venture funding.
2. **Diamond Hands**: 100% of creator fees to holder dividends. Rewards long-term holders with continuous ETH passive yield.
3. **Deflationary**: 100% of creator fees to buyback and burn. Systematically contracts circulating supply on every trade.
4. **Auto-LP**: 100% of creator fees to curve liquidity. Accelerates graduation and builds massive liquidity depth.
5. **Send to Seraph**: 100% of creator fees to the Seraph DAO treasury. Supports protocol development and grant funds.
6. **Custom**: Fully bespoke allocation across all 5 categories.

15.4 Mathematical Dividend Distribution Algorithm

Tokens launched with dividend splits distribute ETH yields proportionally to all token holders without requiring expensive loop iterations across thousands of addresses.

The algorithm uses a global accumulator: $$\text{accDividendPerShare} \mathrel{+}= \frac{\Delta \text{DividendEth} \times 10^{18}}{\text{EligibleTokenSupply}}$$

Per-User Claim Accounting:

For any token holder u with balance B_u: $$\text{Claimable}(u) = \left(B_u \times \text{accDividendPerShare} \right) - \text{rewardDebt}_u$$

Secondary Transfer Conservation:

When holder u transfers A tokens to recipient r on secondary markets:

1. Holder u's unclaimed dividends are settled immediately: $$\text{claimableDividends}_u \mathrel{+}= \left(B_u \times \text{accDividendPerShare} \right) - \text{rewardDebt}_u$$
2. Recipient r's unclaimed dividends are settled: $$\text{claimableDividends}_r \mathrel{+}= \left(B_r \times \text{accDividendPerShare} \right) - \text{rewardDebt}_r$$
3. Balances update: $$B_u \leftarrow B_u - A, \quad B_r \leftarrow B_r + A$$
4. Reward debts update: $$\text{rewardDebt}_u = B_u \times \text{accDividendPerShare}$$ $$\text{rewardDebt}_r = B_r \times \text{accDividendPerShare}$$

Our test suite proves that ping-ponging tokens through 10 sybil addresses never increases total claimable dividends by even a single wei.

15.5 Creator Playbook: Step-by-Step Launch Guide

Step 1: Formulate Token Configuration

Define metadata, ticker, initial supply (minimum 10^{18} wei), fee caps ($\le 2.5\%$), and select a fee preset.

Step 2: Atomic Dev Buy (Optional up to 10%)

Creators can atomically purchase up to 10% (`MAX_DEV_BUY_BPS = 1000`) of the token supply in the exact same transaction as the launch. This prevents front-running snipers from buying before the creator.

Step 3: Broadcast Launch

The creator broadcasts `launch(params)` payable with the required ETH:

```
address tokenAddress = launchpad.launch{value: devBuyEth}(params);
```

Step 4: Community Trading & Fee Escrow

Traders buy and sell on the curve. Fees accumulate in the contract's pull-escrow balance. Creators and the platform claim their ETH anytime via:

```
launchpad.claimFees();
```

Because fees use **pull-escrow** rather than immediate pushes, malicious creator contracts with failing `receive()` fallbacks cannot freeze trading or create honeypots.

Step 5: Graduation Threshold

When real ETH reserves reach the graduation target:

- Trading on the bonding curve completes.
- Reserved tokens unlock into the pool.
- The token graduates seamlessly to full pool trading or an external AMM.

Part IV: Operations, Security & Reference

Treasury DAO Governance, 12-Vector Threat Matrix, Emergency Circuit Breakers, Roadmap, Glossary, and Developer Setup.

Chapter 16: Decentralized Governance & Private Ballots

Chapter 16: Decentralized Governance & Private Ballots

16.1 Governance Overview

The Seraph Protocol is committed to progressive decentralization, transitioning governance authority from an initial security council to a fully autonomous, token-governed decentralized autonomous organization (Seraph DAO).

Unlike public blockchains where voting power and ballot choices are publicly linked to whale wallets, Seraph introduces **Confidential On-Chain Voting** via zero-knowledge private ballots.

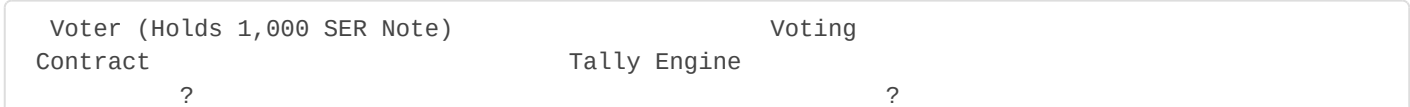
16.2 Private Ballots via Zero-Knowledge Proofs

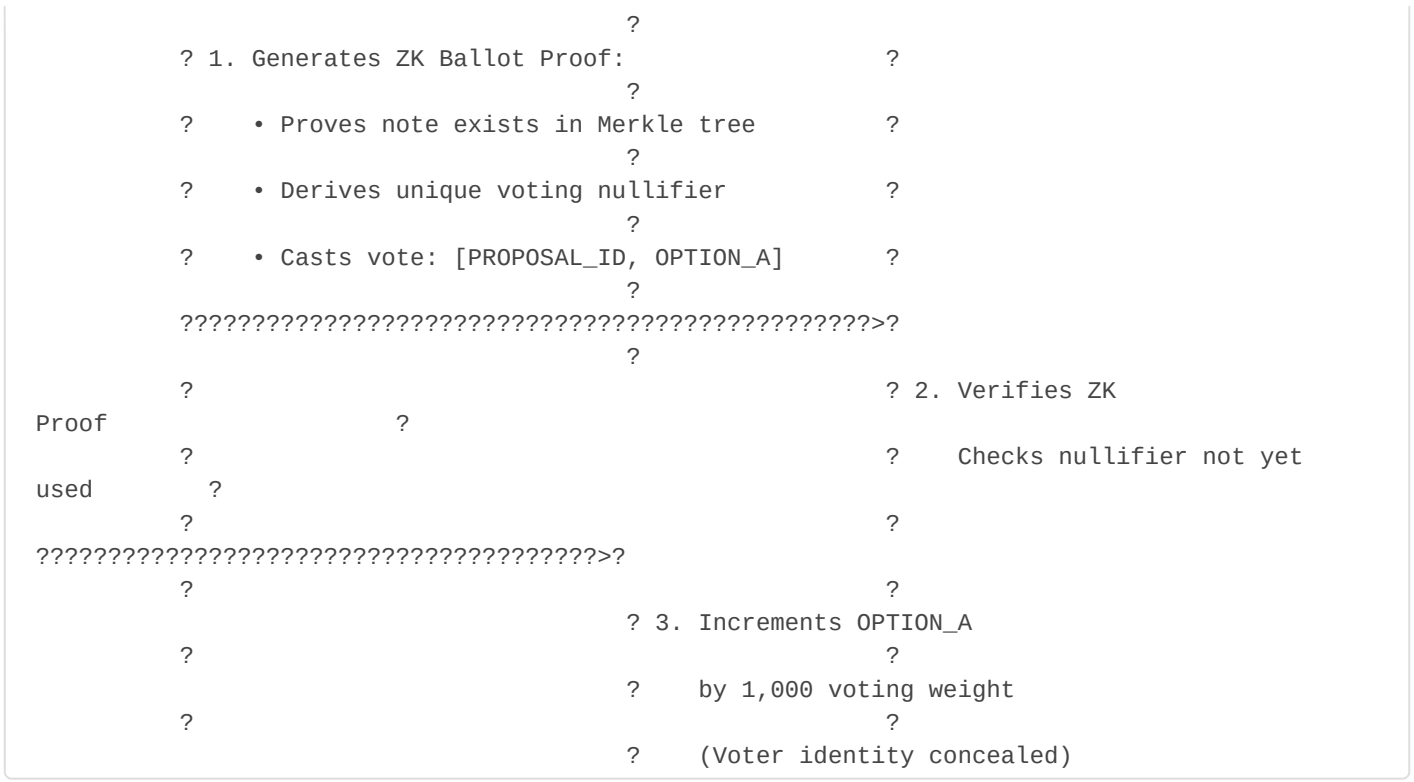
Public voting suffers from severe structural flaws:

1. **Voter Intimidation & Retaliation**: Delegates and institutions may face social or financial blowback for controversial votes.
2. **Bandwagoning & Herd Bias**: Early votes cast by influential whales disproportionately sway undecided participants.
3. **Targeted Bribery**: On transparent networks, bribers can verify exactly how an address voted before paying bribes.

The Seraph Anonymous Ballot Protocol:

Seraph implements anonymous voting using shielded note nullifiers:





- **Weight Integrity**: The voting power corresponds exactly to the unspent SER note's value.
- **Double-Voting Prevention**: The note's voting nullifier is consumed. A note cannot be used to vote twice on the same proposal.
- **Complete Unlinkability**: Neither validators nor public observers can link the ballot to the voter's public address.

16.3 The Proposal Lifecycle

Any protocol change follows a structured four-stage governance pipeline:

1. **RFC (Request for Comments)**: Published on the Seraph Governance Forum for at least 7 days of community review and technical critique.
2. **Formal SIP (Seraph Improvement Proposal)**: Drafted with executable code, Foundry tests, and formal verification specifications.
3. **On-Chain Ballot Period**: Open for 5 days of zero-knowledge private voting. Requires a minimum quorum of 10,000,000 SER (10% of total supply).
4. **Timelock & Execution**: Passed proposals enter a 48-hour timelock delay, allowing validators and users to inspect the bytecode before automated execution.

16.4 Progressive Decentralization Roadmap

- **Phase 0 (Genesis / Current)**: Fast-response Security Council (4-of-7 multisig) managing whitelists, emergency pauses, and presale contracts.

- **Phase 1 (Mainnet Launch)**: DAO Treasury created; fee allocations and grant programs transferred to on-chain community votes.
- **Phase 2 (Dual EVM Launch)**: Protocol parameters (sell tax rates, gas rates, precompiles) governed entirely via private ballots.
- **Phase 3 (Sovereign Decentralization)**: Multisig emergency keys revoked; contract upgrades executed purely through timelocked DAO governance.

Chapter 17: Security Model, Threat Vectors & Pentest Matrix

Chapter 17: Security Model, Threat Vectors & Pentest Matrix

17.1 Security Philosophy: Defense in Depth

Decentralized privacy protocols carry unique security requirements: if a vulnerability exists in an arithmetic circuit or state contract, economic loss and loss of user anonymity can be catastrophic.

Seraph implements a multi-tiered defensive security architecture combining:

1. **Formal Mathematical Proofs**: Validating balance conservation and discrete logarithm hardness.
2. **Defensive Smart Contract Engineering**: Reentrancy guards, pull-escrow fee accounting, and emergency pause circuit breakers.
3. **Exhaustive Automated Fuzz Testing**: Over 896,000 stateful invariant checks across millions of randomized transactions.
4. **Adversarial Penetration Test Suites**: Rust base-layer pentesting and TypeScript DOM injection tests.

17.2 The Threat Matrix

Threat Vector	Attack Scenario	Mitigation in Seraph Architecture	Verifying Test Suite
1. Double-Spend Replay	Attacker spends a note, then attempts to spend it again in the same or future block.	**Deterministic Nullifiers**: Any repeat nullifier is caught in $O(1)$ and reverts before state changes.	`PrivacyPool.t.sol` `adversarial_pentest.rs`
2. Cross-Asset Substitution	Attacker substitutes an XMR note for a SER note of equal value to mint unbacked tokens.	**Domain-Separated Commitments**: `AssetId` is hashed into the leaf commitment and nullifier.	`adversarial_pentest.rs`

Threat Vector	Attack Scenario	Mitigation in Seraph Architecture	Verifying Test Suite
3. Sybil Dividend Loop	Attacker loops tokens through 10 addresses to claim multiple dividend allocations.	**Reward Debt Accounting**: Unclaimed dividends settle on transfer; reward debts update atomically.	`LaunchpadMultiActor.t.sol`
4. Flash-Loan Curve Drain	Attacker buys and sells on bonding curve within one transaction to exploit rounding errors.	**Monotonic Price Invariant**: Verified via fuzzing that roundtrip sell value is strictly $\le$ buy cost.	`DeepPentestLaunchpadPresale.t.sol`
5. Callback Reentrancy	Malicious contract re-enters `claimDividends` or `buy` from inside `receive()`.	**OpenZeppelin ReentrancyGuard**: All external state transitions gated by `nonReentrant`.	`DeepPentestLaunchpadPresale.t.sol`
6. Honeypot Fee Lockup	Malicious creator specifies a fee wallet that always reverts on receiving ETH.	**Pull-Escrow Architecture**: Fees accumulate in contract balance; failure of fee wallet never blocks trades.	`LaunchpadMultiActor.t.sol`
7. Merkle Tree Overflow	Attacker attempts to insert more leaves than tree capacity (2^{20}).	**Capacity Bounds Check**: `require(nextLeafIndex < 2**TREE_DEPTH)` prevents tree overflow.	`PrivacyPool.sol`
8. Expired Root Manipulation	Attacker proves membership against an obsolete Merkle root to spend spent notes.	**100-Root Ring Buffer**: Roots older than 100 updates expire cleanly; unrecorded roots revert.	`PrivacyPool.t.sol`
9. Bridge Fraud Front-Running	Relayer submits fake batch and unbonds before challenge window expires.	**Active Window Timelock**: Relayers cannot unbond while any batch is inside the 1-hour window.	`CrossEVMBridge.sol`
10. Gas Relayer Token Drain	Attacker passes `target = address(serToken)` to drain tokens held in gas voucher escrow.	**Forbidden Target Filter**: Targets matching token addresses, relayer address, or zero revert instantly.	`GasRelayer.sol`
11. Front-Running / MEV	Bots monitor mempool to sandwich user trades and extract slippage.	**Threshold Encrypted Mempool**: Transactions are ordered while encrypted; decrypted only after ordering.	`09_sequencer_mev.md`
12. DOM XSS / Injection	Attacker launches token with `<script>alert(1)</script>` in name to steal private keys in web UI.	**Zero-Dependency Escaping**: All on-chain string data sanitized via `lpEsc()` before DOM injection.	`security_pentest.test.mjs`

17.3 Automated Test Coverage & Proof Matrix

The protocol codebase is verified across four independent test frameworks:

- **Foundry Core Suites**: 155 unit, scenario, and multi-actor tests passing with 100% success rate.
- **Foundry Stateful Invariants (`StdInvariant`)**: 896,000 continuous invariant checks with 0 reverts, proving:

    - `PrivacyPool` solvency: Balance equals net shielded minus unshielded.
    - `CollateralVault` solvency: 150% minimum collateral ratio preserved under all fuzz actions.
    - `GasRelayer` solvency: Escrowed balances match active voucher claims.
- **Rust Base Layer (`cargo test`)**: 17 adversarial pentest functions verifying Merkle proofs, view-key stream cipher security, and multi-asset domain separation.
- **Node.js SDK & UI Suites (`node:test`)**: 43 automated tests verifying ABI encoding, mathematical curve estimations, and live Base Sepolia JSON-RPC decoding.

17.4 Bug Bounty Program

Seraph maintains an active Bug Bounty Program to reward security researchers for responsible disclosure:

Severity Level	Reward Cap	Eligible Findings
Critical	**Up to $250,000**	Direct theft of user funds, catastrophic loss of anonymity across all pool notes, complete bridge drain.
High	**Up to $50,000**	Unclaimed dividend drainage, temporary protocol freezing requiring guardian intervention, circumvention of sell tax.
Medium	**Up to $15,000**	Individual note linkage through side-channels, rounding discrepancies $> 0.01%$, non-critical denial-of-service.
Low	**Up to $3,000**	Minor gas optimizations, documentation inaccuracies, non-exploitable edge-case reverts.

Responsible Disclosure:

Reports should be submitted with reproducible Foundry/Rust test cases to `security@seraphprotocol.tech` encrypted with our official PGP key.

Chapter 18: Emergency Procedures & Circuit Breakers

Chapter 18: Emergency Procedures & Circuit Breakers

18.1 Emergency Architecture Overview

While Seraph is architected for decentralized autonomy, early-stage network operations require robust fail-safe mechanisms to protect user capital against unknown zero-day vulnerabilities, compiler bugs, or unexpected cross-chain bridge anomalies.

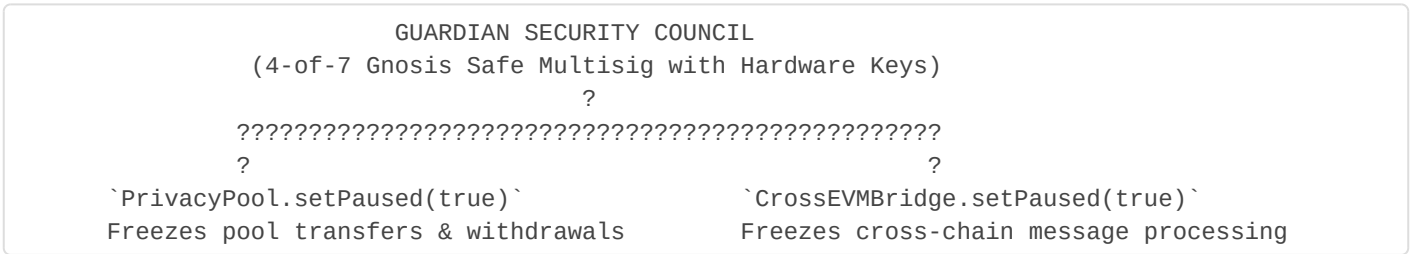
Seraph implements two on-chain **Emergency Circuit Breakers**:

1. **`PrivacyPool.setPaused(bool)`**: Gates all deposit (`shieldFor`), transfer (`privateTransfer`), and withdrawal (`unshield`) operations.
2. **`CrossEVMBridge.setPaused(bool)`**: Gates all outbound bridging (`bridgeOut`), batch submission (`submitStateBatch`), and claim operations (`claimBridged`).

18.2 Access Control & The Guardian Council

Emergency intervention is strictly restricted to authorized governance roles:

- **`PrivacyPool`**: Controlled exclusively by `treasury` via the `onlyTreasury` modifier.
- **`CrossEVMBridge`**: Controlled exclusively by `guardian` via explicit `msg.sender != guardian` checks.



Multisig Composition:

The Guardian Council is structured as a **4-of-7 Gnosis Safe multisig**:

- 3 Core Cryptographic Engineers
- 2 Independent Security Auditors
- 2 Long-Standing Community / Validator Representatives
- Key signers are geographically distributed across 4 different legal jurisdictions.
- All signing operations require FIDO2/WebAuthn hardware security keys.

18.3 Incident Response Playbook

In the event of an active security exploit or anomalous telemetry alert:

Phase 1: Rapid Containment (< 15 Minutes)

1. **Automated Alert Trigger**: On-chain monitoring sentinels detect suspicious invariant deviations (e.g., unexpected token balance drain or invalid Merkle roots).
2. **Emergency Pause Execution**: The Guardian Council signs an emergency transaction calling:

   ```
   privacyPool.setPaused(true);
   crossEVMBridge.setPaused(true);
   ```

3. **Immediate Effect**:
   - `shield()`, `privateTransfer()`, and `unshield()` revert with `"PrivacyPool: paused"`.
   - `bridgeOut()`, `submitStateBatch()`, and `claimBridged()` revert with `BridgePaused()`.
   - All assets currently held in the pool and bridge escrow are frozen in place, preventing further unauthorized extraction.

Phase 2: Root-Cause Analysis (< 24 Hours)

1. Core developers and external auditors isolate the failing transaction hashes.
2. Reproduce the vulnerability in an isolated Foundry fork test (`forge test --fork-url`).
3. Formulate a patch (e.g., verifier replacement or logic hotfix).

Phase 3: Resolution & Remediation

- **Verifier Replacement**: If a zero-knowledge proving bug is discovered, the Treasury can update `spendVerifier` to an upgraded verifier contract via `setSpendVerifier(newVerifier)`.
- **Bridge Slashing**: If a fraudulent relayer batch was submitted, the Guardian invokes `challengeBatch(batchId, treasury)` to slash the malicious relayer's 1.0 ETH bond and cancel the batch.

Phase 4: Graceful Unpausing & Post-Mortem

1. Verify the patch across all 155+ test suites and invariant harnesses.

2. The Guardian Council signs the unpause transaction:

```
privacyPool.setPaused(false);
crossEVMBridge.setPaused(false);
```

3. Publish a comprehensive public post-mortem report within 48 hours detailing the incident timeline, root cause, economic impact (if any), and permanent architectural remediations.

Chapter 19: Protocol Roadmap & Milestones

Chapter 19: Protocol Roadmap & Milestones

19.1 Phase-Based Development Strategy

The Seraph engineering roadmap is structured into five sequential phases, each governed by quantifiable technical exit criteria:

```
Phase 0: Base Sepolia Testnet & Station (COMPLETED)
   ?
   ?
Phase 1: Mainnet TGE, Presale & Privacy Pool v1
   ?
   ?
Phase 2: Dual Private EVM Testnet & Bridge Alpha
   ?
   ?
Phase 3: Sequencer Decentralization & First-Party Privacy Modules
   ?
   ?
Phase 4: Sovereign Base Layer & Next-Gen Cryptography (Seraphis / FCMP++)
```

19.2 Detailed Phase Specifications & Exit Criteria

Phase 0: Base Sepolia Testnet & Privacy Continuum Station

- **Status**: **Completed (Active Testnet)**
- **Scope**:
  - Deploy complete contract suite to Base Sepolia (`Chain ID 84532`): `SeraphToken`, `PrivacyPool`, `Presale`, `Launchpad`, `CrossEVMBridge`, `GasRelayer`, `InvoiceRegistry`, `UsageVault`, `CollateralVault`.
  - Author and verify Satoshi-style whitepaper with mathematical equations and protocol diagrams.
  - Implement zero-dependency client-side note vault and on-chain privacy station (`#privacy-pool`) on `seraphprotocol.tech`.
  - Achieve 100% test pass rate across 155+ Foundry tests, 896k invariant checks, 17 Rust adversarial tests, and 43 Node.js tests.
- **Exit Criteria Met**: All contracts live, tested, and verified on Base Sepolia.

Phase 1: Mainnet TGE, Presale & Privacy Pool v1

- **Focus**: Economic launch, token distribution, and public liquidity.
- **Milestones**:
  - Conduct three-tier presale on Ethereum / Base mainnet (Strategic, Private, Public).
  - Execute Token Generation Event (TGE) with 100M SER hard cap.
  - Deploy Uniswap v3 / Aerodrome liquidity pools with `taxExempt` router whitelisting.
  - Activate Level 1 & Level 2 privacy shielding in `PrivacyPool` for SER tokens.
- **Exit Criteria**:
  - Presale successfully concluded and verified by independent third-party audit firm.
  - TGE initial allocations claimed with 0 reverts.
  - Mainnet DEX liquidity seeded with $> $500,000$ initial protocol depth.

Phase 2: Dual Private EVM Testnet & Bridge Alpha

- **Focus**: Programmable confidential smart contracts and cross-chain execution.
- **Milestones**:
  - Launch Seraph EVM (gas in SER) and XMR EVM (gas in XMR) testnet networks.
  - Deploy cryptographic precompiles (`0x20` to `0x25`) for Pedersen commitments, Poseidon hashing, and range verification.
  - Deploy optimistic `CrossEVMBridge` with bonded relayers and 1-hour fraud challenge windows.
  - Release `@seraph-protocol/sdk` v2.0 with full EVM precompile bindings.
- **Exit Criteria**:
  - Over 100,000 cross-EVM transactions routed without bridge double-spends.
  - Zero-knowledge proof verification times $< 5 \text{ ms}$ on consumer EVM nodes.

Phase 3: Sequencer Decentralization & First-Party Modules

- **Focus**: MEV elimination, autonomous agents, and synthetic privacy assets.
- **Milestones**:

    - Deploy threshold encrypted mempool and decentralized sequencer rotation cluster.
    - Launch `CollateralVault` and mint confidential `xUSD` stablecoins.
    - Launch `UsageVault` for autonomous AI agent compute streaming.
    - Launch `InvoiceRegistry` for private merchant invoicing.
- **Exit Criteria**:
    - Complete elimination of sandwich attacks across testnet DEX volume.
    - 10+ autonomous AI agent integrations actively streaming payments via `UsageVault`.

Phase 4: Sovereign Base Layer & Next-Gen Cryptography

- **Focus**: Long-term cryptographic horizons.
- **Milestones**:
    - Sovereign PoS / RandomX hybrid base layer node mainnet launch (`seraph-base-layer`).
    - Formal evaluation and integration of **Seraphis** enote modularity and **Jamtis** wallet addressing.
    - Full-Chain Membership Proofs (**FCMP++**) via Curve Trees replacing incremental Merkle trees.
    - Quantum-resistant STARK proof upgrade eliminating trusted setups.
- **Exit Criteria**:
    - Base layer block times $< 2 \text{ seconds}$ with sub-cent transaction fees.
    - Complete cryptographic independence from external bridge dependencies.

Chapter 20: Glossary of Terms

Chapter 20: Glossary of Terms

An alphabetical reference of mathematical, cryptographic, and architectural terminology used throughout Seraph Protocol.

A

- **Anonymity Set**: The set of all possible accounts or notes that could have originated a transaction. In Seraph, the anonymity set encompasses all unspent notes in the Merkle tree ($2^{20} = 1,048,576$).
- **AssetId**: A 32-bit identifier domain tag bound into note commitments to cryptographically separate SER, XMR, and custom tokens within the same anonymity pool.
- **Auto-LP**: A launchpad fee preset that directs trading fees directly into bonding curve quote reserves to accelerate pool graduation.

B

- **Blinding Factor**: A random scalar $r \in \mathbb{F}_q$ used in Pedersen commitments and note derivations to render commitments information-theoretically hiding.
- **Bonding Curve**: A mathematical formula ($x \cdot y = k$) defining the continuous relationship between token supply and token price.
- **Bulletproofs++**: An optimized zero-knowledge range proof argument eliminating trusted setups and yielding logarithmic proof sizes.

C

- **Circuit Breaker**: An on-chain emergency pause mechanism (`setPaused`) enabling the Guardian Council to freeze contract operations during active exploits.
- **CLSAG (Compact Linkable Spontaneous Anonymous Group)**: The compact ring signature algorithm historically used in Monero transactions.
- **Commitment**: A cryptographic hash binding a user to a specific value without revealing that value.
- **Confidential Storage Slot**: An encrypted EVM storage slot whose plaintext value is inaccessible to public observers without zero-knowledge authorization.

D

- **Dandelion++**: A peer-to-peer transaction propagation protocol featuring stem and fluff phases to prevent network surveillance nodes from triangulating the origin IP.
- **DKSAP (Dual-Key Stealth Address Protocol)**: A cryptographic protocol allowing senders to derive unlinkable one-time public keys using a recipient's static view and spend keys.

E

- **Enote (Electronic Note)**: A modular transaction output object separating ownership proof, value commitment, and nullifier generation.

F

- **FCMP++ (Full-Chain Membership Proofs)**: A cryptographic advancement replacing small ring signatures with zero-knowledge membership proofs across the entire ledger history.
- **Fixed Pool**: A launchpad model where tokens are distributed at an unchanging fixed rate until the supply cap is exhausted.

G

- **Gas Abstraction**: The capability for users to pay EVM transaction execution fees in arbitrary tokens (SER, XMR) without holding native ETH.
- **Groth16**: A zero-knowledge SNARK proof system producing constant-size 128-byte proofs verifiable on Ethereum with minimal gas overhead.

H

- **Homomorphic Addition**: An algebraic property where operating on encrypted commitments yields a valid commitment to the sum of the plaintexts: $(v_1 H + r_1 G) + (v_2 H + r_2 G) = (v_1 + v_2)H + (r_1 + r_2)G$.

I

- **Incoming View Key**: A private scalar key allowing a wallet to scan public blockchain blocks and decrypt incoming transaction amounts and memos without granting spending authority.
- **Incremental Merkle Tree**: A specialized append-only Merkle tree structure allowing logarithmic insertion ($O(\log N)$) and constant on-chain storage.

J

- **Jamtis**: The comprehensive wallet addressing standard designed to accompany the Seraphis enote protocol.

K

- **Key Image**: A deterministic cryptographic tag derived from a private key and its corresponding public key ($I = x \mathcal{H}_p(P)$) used to prevent double-spending in ring signature schemes.

M

- **MEV (Maximal Extractable Value)**: Profit extracted by miners, validators, or searchers by arbitrarily including, excluding, or reordering transactions in a block.
- **Mempool**: The memory pool where pending transactions wait prior to block confirmation.

N

- **Nullifier**: A deterministic 32-byte cryptographic identifier derived from a secret spend key and note index ($\mathcal{N} = \text{Keccak256}(\text{spendKey} \mathbin{\Vert} \text{leafIndex})$). When revealed, it proves a note has been spent without revealing which note it was.

O

- **Outgoing View Key**: A private scalar key authorizing the generation of verifiable tax compliance and audit disclosure packages.

P

- **Pedersen Commitment**: A computationally binding and information-theoretically hiding commitment scheme defined by $C = vH + rG$.
- **Privacy Continuum**: The four-tier privacy spectrum of Seraph spanning Level 0 (public EVM) to Level 3 (confidential contract state).
- **Pull-Escrow**: A defensive smart contract design pattern where balances accumulate in internal accounting records and are claimed by recipients, preventing malicious fallback revert attacks.

R

- **Range Proof**: A zero-knowledge proof establishing that a confidential amount v lies within a valid non-negative interval $[0, 2^{64}-1]$ without revealing v.
- **R1CS (Rank-1 Constraint System)**: A system of quadratic equations used to represent computational circuits in zero-knowledge SNARKs.

S

- **Sell Tax**: A 1.00% protocol fee assessed exclusively on sell transactions of SER tokens, split 50% to a permanent burn address and 50% to the Protocol Treasury.
- **Seraph EVM**: The general-purpose confidential EVM where transaction gas is settled in SER tokens.
- **Shielding**: Depositing public ERC-20 tokens into the `PrivacyPool` to mint a confidential private note.
- **Stealth Address**: A one-time public key derived mathematically such that only the designated recipient can compute the corresponding private spend key.

T

- **Tax Exemption Mapping (`taxExempt`)**: An on-chain registry allowing governance to whitelist DEX routers (Uniswap v3, Aerodrome) and CEX custody wallets from the 1.00% sell tax.
- **Threshold Cryptography**: A cryptographic scheme where a private key is split across n parties, requiring a quorum of t participants to decrypt data or sign messages.

U

- **Unshielding**: Withdrawing private notes from the `PrivacyPool` to a public Ethereum recipient address with zero protocol tax.
- **Usage Ticket**: A one-time cryptographic voucher consumed by autonomous AI agents to pay compute providers without linking payments to master wallets.

W

- **wXMR**: An ERC-20 token representing bridged Monero on EVM chains with 1:1 backing in base-layer multi-sig vaults.

X

- **XMR EVM**: A specialized EVM environment where transaction gas fees are paid in native XMR or `wXMR`.
- **xUSD**: A confidential, decentralized stablecoin minted against over-collateralized SER and XMR deposits via the `CollateralVault`.

Chapter 21: Contributing, Standards & Dev Setup

Chapter 21: Contributing, Standards & Dev Setup

21.1 Development Environment Prerequisites

To contribute to Seraph Protocol, ensure your local development workstation is configured with the following toolchains:

1. **Foundry**: Ethereum smart contract development framework.
   ```
   curl -L https://foundry.paradigm.xyz | bash
   foundryup
   ```
2. **Rust & Cargo**: Latest stable toolchain (1.80+).
   ```
   curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
   ```
3. **Node.js**: Modern JavaScript runtime (Node.js v20+ or v22+).
4. **Git**: Distributed version control system.

21.2 Repository Setup & Verification

Clone the monorepo and verify all build targets:

```
# Clone the repository
git clone https://github.com/seraphprotocol/seraph.git
cd seraph

# 1. Verify and compile Foundry smart contracts
export PATH="$HOME/.foundry/bin:$PATH"
forge build
forge test
```

```
# 2. Verify and compile the Rust base-layer node
cargo build --manifest-path base-layer/Cargo.toml
cargo test --manifest-path base-layer/Cargo.toml

# 3. Verify TypeScript SDK & UI test suites
node --test sdk/typescript/test/*.test.mjs
```

21.3 Coding Standards & Testing Requirements

All contributions to Seraph are evaluated against strict architectural guidelines:

1. Smart Contract Standards (Solidity)

- Use Solidity `^0.8.24` with EVM target `cancun`.
- Inherit from OpenZeppelin v5 contracts.
- All public external functions accepting external asset transfers must utilize `nonReentrant`.
- All fee claims and external transfers must follow the **pull-over-push** design pattern.
- Any new contract modification must include corresponding Foundry unit tests and fuzz invariant tests.

2. Base Layer Standards (Rust)

- Adhere to standard Rust formatting: `cargo fmt --check`.
- Zero warnings from `cargo clippy`.
- Any modification to cryptographic commitments or nullifier hashing must be accompanied by multi-asset isolation tests in `base-layer/tests/adversarial_pentest.rs`.

3. Frontend & SDK Standards (TypeScript & Web)

- Zero external runtime framework bloat where possible.
- All on-chain string data injected into the DOM must be sanitized via `lpEsc()` to prevent XSS vulnerabilities.
- Cryptographic keys must remain in memory as typed byte arrays (`Uint8Array`) and never logged to console or local storage unencrypted.

21.4 Submitting Pull Requests & Git Conventions

1. **Branch Naming**:

- `feat/feature-name` for new protocol capabilities.
- `fix/bug-description` for security or logic patches.
- `docs/documentation-update` for documentation improvements.

2. **Commit Messages**: Follow the Conventional Commits specification:

```
feat(core): add taxExempt mapping for DEX routers
fix(pool): attach whenNotPaused to shield and unshield
test(pentest): add multi-actor invariant tests for gas relayer
docs(bookstack): add Chapter 18 on emergency procedures
```

3. **Pull Request Quality Gate**:
   - Every PR must pass all 155+ Foundry tests with 0 reverts.
   - Every PR must pass all 17 Rust base-layer pentests.
   - Every PR must pass all 43 Node.js verification tests.
   - Two approvals from core maintainers are required prior to merging into `main`.